



UNIVERSITY
INSTITUTE
OF LISBON

A Cross-Layer SDN Orchestration Architecture for Stateful Edge Services

Pedro Ojo Oriakhi

Master's in Telecommunications and Computer Engineering

Supervisors:

Ph.D. Professor José Moura, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Ph.D. Professor Pedro Ramos, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Month, Year

[This page is intentionally left blank.]



TECHNOLOGY
AND ARCHITECTURE

Department of Information Science and Technology

A Cross-Layer SDN Orchestration Architecture for Stateful Edge Services

Pedro Ojo Oriakhi

Master's in Telecommunications and Computer Engineering

Supervisors:

Ph.D. Professor José Moura, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Ph.D. Professor Pedro Ramos, Assistant Professor,
Iscte - Instituto Universitário de Lisboa

Month, Year

[This page is intentionally left blank.]

Write here your dedication.

[This page is intentionally left blank.]

Acknowledgments

Write here your acknowledgements and, if applicable, any grants or sources of funding.

[This page is intentionally left blank.]

Resumo

Palavras-Chave:

[This page is intentionally left blank.]

Abstract

Write here the abstract in English. There is a 250 word limit.

Keywords: *Keywords; separated; by; semicolons*

[This page is intentionally left blank.]

Contents

Acknowledgments	iii
Resumo	v
Abstract	vii
List of Figures	xi
List of Tables	xiii
List of Acronyms	xv
Chapter 1. Introduction	1
1.1. Context, Motivation, and Problem Statement	1
1.2. Objectives	4
1.3. Research Questions	4
1.4. Research Methodology	5
1.5. Main Contributions of this Dissertation	7
1.6. Dissertation Structure	7
Chapter 2. Background and Related Work	9
2.1. Review Methodology	9
2.2. Platform Technology Background	10
2.2.1. Edge Computing	11
2.2.2. SDN and OpenFlow	11
2.2.3. Containerisation and Docker	12
2.2.4. NoSQL and MongoDB	12
2.3. Monitoring & Telemetry	13
2.4. Auto Scaling	15
2.5. Load Balancing and Readiness Admission on SDN	17
2.6. SDN-Based Orchestration Platforms	18
2.7. Synthesis and Research Gaps	20
Chapter 3. Architecture and Design of the Proposed Solution for Edge Services	23
3.1. Design Requirements	23
3.2. Architecture	24
3.2.1. Two-Domain Organisation	24
3.2.2. Virtual IP Data Paths	25
	ix

3.2.3. Control and Observation Paths	26
3.3. Distributed Elastic Allocation of Resources	27
3.3.1. Scaling Policy	28
3.3.2. Compute	30
3.3.3. Storage	31
3.4. Monitoring and Decision Engine	31
3.5. Control Workflow	34
Chapter 4. Implementation of Proposed Solution	37
4.1. Experimental infrastructure	37
4.2. Distributed Elastic Allocation of Resources	37
4.2.1. Compute	37
4.2.2. Storage	37
4.3. Monitoring and Decision Engine	37
4.4. Control Workflow	37
4.5. Implementation Validation	37
Chapter 5. Experimental Results	39
5.1. Experimental Setup	39
5.1.1. Testbed and Instrumentation	39
5.1.2. Demand Model and Campaign Workloads	40
5.1.3. Metrics and Outcome Definitions	41
5.1.4. Campaign Designs and Analysis	42
5.2. Telemetry Delivery Semantics (RQ1)	44
5.3. Bottleneck-Aware Scaling Action (RQ2)	46
5.4. Readiness Admission to Traffic (RQ3)	46
5.5. Discussion: Synthesis Across Interfaces	48
Chapter 6. Conclusions and Future Work	49
6.1. Conclusions	49
6.2. Research Contributions	49
6.3. Limitations	49
6.4. Future Research Directions	49
References	51

List of Figures

Figure 1.1	Global internet traffic between 2020 and 2025. Source: Statista, based on ITU data [4].	1
Figure 1.2	The demand-to-capacity chain: from a demand shift to usable capacity, with the three interfaces studied in this dissertation highlighted.	3
Figure 1.3	Design Science Research Methodology	5
Figure 2.1	Software-Defined Networking architecture: the control plane is decoupled from the data plane and programs the forwarding devices through the OpenFlow protocol.	12
Figure 2.2	MongoDB replica set: a primary member serving the clients and secondary members to which the data is replicated.	13
Figure 2.3	Load balancing in an SDN network: the controller installs forwarding rules that distribute client traffic among the available backends.	17
Figure 3.1	Architecture of the proposed platform: two network domains whose clients, data planes, edge servers, storage tiers, and aggregators are coordinated by peer SDN controllers, with host-local control paths and an emulated wide-area data link between the domains.	24
Figure 3.2	Double-VIP routing sequence: a client request addressed to <code>VIP_SERVER</code> triggers backend selection in the controller, and the edge server's storage reads through <code>VIP_DATA</code> trigger the selection of a replica-set member.	26
Figure 3.3	Control and observation paths: edge servers and storage members publish observations to the per-domain telemetry aggregator, whose windowed summaries reach both peer controllers through a configurable delivery interface.	27
Figure 3.4	Elastic allocation of resources: each tier is scaled on its own observation signals, with compute replicas provisioned and drained and storage members added and removed through replica-set membership, both actions issued by the controller over the infrastructure-management path.	28
Figure 3.5	The per-window evaluation, in the fixed order of Section 3.3.1: (1) the eligibility check, where the window is skipped when the tier is ineligible;	

(2) the degradation score $s(w)$, computed from the normalised signals;
(3) the scale-up condition: a scale-up request is emitted and the window history is cleared when at least R of the last W eligible windows reach the adaptive threshold $\theta(k)$; and (4) the idle condition, tested otherwise: idleness is tracked, excluding indeterminate windows, and a scale-down request is emitted once it spans $R_{\downarrow}$ of the last $W_{\downarrow}$ windows. Every remaining branch returns to the next telemetry window. 33

Figure 3.6 The control workflow, drawn as a UML diagram. The demand-to-capacity chain runs across three contexts in the same process: the packet context, the observation context, and the action context. The contexts do not block one another and they exchange state through shared structures (the telemetry state, the action queue, the lifecycle state, and the routing pools). The numbered stages match the chain in Figure 1.2 and show the provisioning direction. Capacity removal follows the same steps in reverse, starting with the drain. The aggregator is omitted between the observations sent by the replicas and the windowed summary delivered to the controller. 35

Figure 5.1 Episode endpoint latency p95 by delivery semantics (served basis; per-run mean of the two domains; seven replicates per arm). Bars show the median across replicates, whiskers the min-max range, and dots the individual runs; the two highest delayed points are the replicates where the base fleet’s absorption margin was consumed. 46

List of Tables

Table 2.1	Search-term families by domain.	10
Table 2.2	Selection criteria.	10
Table 2.3	The three interfaces of the demand-to-capacity chain and the questions they leave open.	21
Table 5.1	Evaluation testbed configuration common to the three campaigns.	39
Table 5.2	Design of the telemetry delivery campaign (RQ1).	42
Table 5.3	Design of the scaling action campaign (RQ2).	43
Table 5.4	Design of the readiness admission campaign (RQ3).	43
Table 5.5	Delivery-semantics arms: usable capacity, episode latency, and outcome classes (medians with min–max ranges over the seven replicates; delivered fraction and timeouts as observed). Usable capacity is the time from episode onset to first new capacity serving traffic; the episode p95 is the served-basis endpoint latency p95 (per-run mean of the two domains).	45
Table 5.6	Pre-registered primary contrasts on the episode endpoint latency p95 (served basis; per-run mean of the two domains). Exact Mann–Whitney p -values (complete enumeration, $n = 7$ per arm); Cliff’s δ computed as $P(x > y) - P(x < y)$ with 95 % bootstrap CI; direction = blocks (of seven) consistent with the predicted order. No multiplicity correction (four pre-registered edges).	46
Table 5.7	Admission-chain intervals and healthy-regime outcomes (intact readiness-event source; direct lifecycle notification versus periodic discovery). Medians over runs and per-backend intervals as marked; exact Mann–Whitney p -values as pre-registered.	48
Table 5.8	Readiness-admission configurations under total event loss (event-loss part of the RQ3 campaign; six blocks, plateau pooled). Admitted = visibility in the selector’s eligible pool; abandoned candidates in parentheses. Plateau tail = slow share over the last 300 s of the plateau; *block 4 is excluded from the fault contrast, its no-fault control tail (4.08 %) breaching the 2 pp floor.	48

[This page is intentionally left blank.]

List of Acronyms

SDN: Software-Defined Networking

WSM: Weighted Sum Model

VIP: Virtual IP

OVS: Open vSwitch

ZMQ: ZeroMQ

DSRM: Design Science Research Methodology

[This page is intentionally left blank.]

CHAPTER 1

Introduction

Edge web services that encompass routing, monitoring and auto-scaling rely on several control loops, and when these loops operate independently the handoffs between them accumulate delays. This accumulation of handoff delays degrades orchestration efficiency and reliability, and may reduce service quality [1, 2, 3].

The remainder of this chapter is organised as follows. Section 1.1 presents the context, motivation and problem statement; Section 1.2 states the objectives; Section 1.3 lists the research questions; Section 1.4 details the methodology used to develop the thesis; Section 1.5 summarises the main contributions; and Section 1.6 outlines the rest of the dissertation.

1.1. Context, Motivation, and Problem Statement

In 2025, global internet traffic across fixed and mobile networks exceeded 7 000 exabytes¹, more than doubling since 2020 [4], as shown in Figure 1.1.

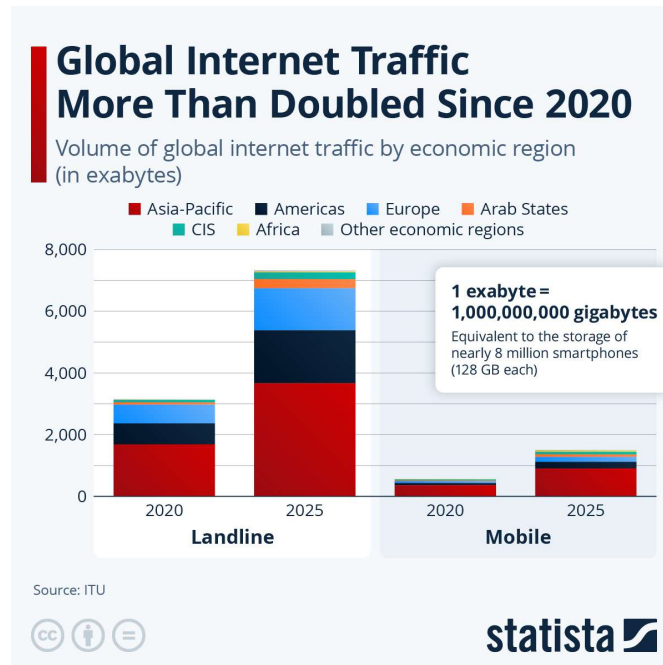


FIGURE 1.1. Global internet traffic between 2020 and 2025. Source: Statista, based on ITU data [4].

¹One exabyte equals 10^9 gigabytes.

This growth has been driven by many technological advancements, some of which have in turn spurred the emergence of real-time interactive and data-intensive applications, among others. Examples include web streaming and cloud gaming, both services where users expect sub-second latency and high-resolution multimedia, highlighting the need for fast and capable data processing. These examples demonstrate the demand for infrastructure able to deliver performance with high availability and reliability at a global scale, regardless of where the end users are located.

Cloud computing has been the backbone that enables these applications to operate at a global scale. It offers automatic scaling, both horizontally and vertically, a pay-per-use model and, among other aspects, a globally distributed offering and availability that have established it as the foundational layer supporting modern applications and services [5].

Yet, in parallel, a growing share of internet traffic now originates from mobile devices, which have become the main gateway to internet access [6]. Much of this access is therefore generated and consumed at the network edge, far from centralised cloud data centres. Edge computing brings the computational resources to the network edge, therefore closer to the end-users, allowing for much lower latency in delivering services and applications to users by leveraging its geo-distributed nature [7, 8]. Beyond lower latency, edge computing’s inherent geo-distributed servers dispersed geographically and close to users relieves bandwidth pressure on the core network, since data can be processed and stored locally, at the edge, for quicker responses, only a subset of it is sent to the cloud [7], freeing up backbone available bandwidth.

Despite these advantages, this geographical distribution also translates into less capacity per location and greater complexity in managing the distributed edge servers. As Satyanarayanan puts it, “the dispersion inherent in edge computing raises the complexity of management considerably” [8, p. 37]. This management complexity stems from the dispersion of the edge servers across many sites, aggravated by considerable variation in user demand over time and by the heterogeneous resource requirements of the requests served. Luo et al. further highlight that, without efficient resource scheduling, this distributed nature leads to resource waste [9]. Resource management therefore becomes increasingly important, since the limited capacity of each location must be adjusted over time to suit the constant fluctuation of user demand.

The resource management problem gains more relevance when the services themselves are stateful. Unlike stateless microservices, which can be replicated freely and absorb traffic immediately, stateful services in a geo-distributed edge deployment depend on data held close to the compute. A content item ingested at one site is served from the database of that site. In distributed deployments, such data is routinely replicated across sites for locality and robustness, but replication is neither instantaneous nor free. A request reaching a server whose local copy is still synchronising must wait for it to catch up, and maintaining copies consumes storage and replication bandwidth [10].

Scale-up alone does not guarantee usable capacity instantly. The orchestration system must detect when data must be brought closer, deliver that information to the decision point, provision the right kind of capacity, and redirect traffic to it. Each of these steps, when executed in a system whose components must communicate with each other over the network, can introduce delay or data loss that degrades orchestration efficiency and, in turn, the quality of service experienced by users.

This sequence forms what this dissertation calls the demand-to-capacity chain, which is the path that an orchestration system takes to convert a change in overall user demand into capacity that serves that demand. A demand shift is first observed by monitoring and telemetry, which deliver the evidence to a decision point. The decision point selects a capacity action, such as scaling compute or scaling storage, then with the action a new capacity is provisioned. Finally the new capacity attains a readiness state and is admitted to traffic, which is the point at which it becomes usable. The chain is therefore a timeline through the orchestration system, and certain steps are separated from the next by an interface at which delay or loss of evidence can accumulate.² Three of these interfaces, more specifically how demand evidence reaches the decision, which capacity action the decision hands over to the infrastructure, and when a ready backend is admitted to traffic, are the subject of this thesis. Figure 1.2 illustrates the chain.

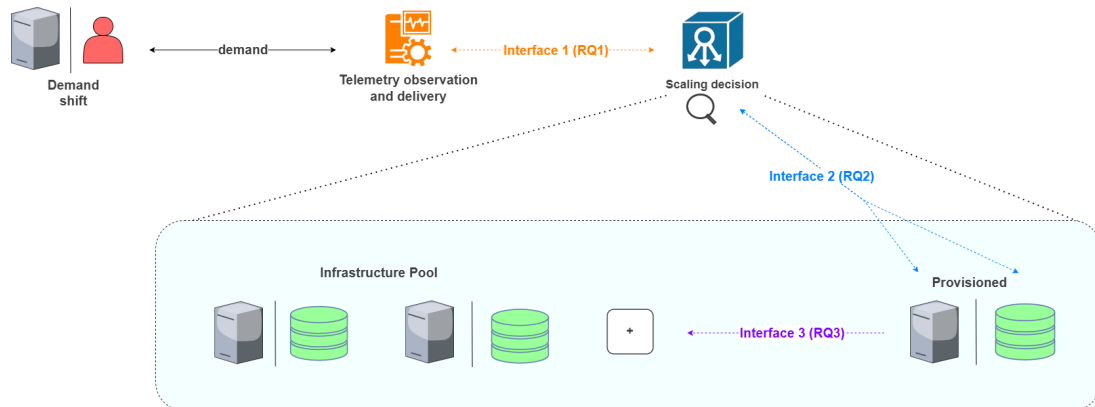


FIGURE 1.2. The demand-to-capacity chain: from a demand shift to usable capacity, with the three interfaces studied in this dissertation highlighted.

The steps described above must be coordinated under the time pressure of an ongoing demand shift. In practice, however, they are commonly performed by separate components. A monitoring system collects metrics, an auto-scaler provisions capacity, and a load balancer redirects traffic, each running under its own control loop. The information needed to respond to demand must therefore cross from one component to the next, and

²Throughout this dissertation, the *demand-to-capacity chain* denotes the full timeline from a demand shift to usable capacity. An *interface* is a boundary between two steps of the chain at which two functions meet. An *handoff delay* is the latency added at an interface when the functions on either side run under independent control loops, that is, without explicit coordination between them. And the *coordination gap* is the accumulated effect of these handoff delays along the chain.

the delays accumulated at these handoffs determine how quickly newly provisioned capacity can actually serve traffic. In the reviewed literature, these handoffs are acknowledged in passing but never isolated or varied as an independent variable [1, 3]. This thesis investigates them directly through two peer Software-Defined Networking (SDN) controllers, one per network domain. Within each controller, telemetry consumption, scaling decisions, and traffic routing run in the same process. Telemetry production and aggregation remain external to the controller, making the delivery of aggregated demand evidence an explicit and configurable interface. Together with explicit lifecycle hooks for capacity action and traffic admission, this arrangement allows each selected interface to be measured under controlled conditions.

Along this chain, three interfaces are studied in a stateful service deployed across two geo-distributed sites: how demand evidence is delivered to the controller (telemetry delivery semantics), which capacity action the controller hands over to the infrastructure in response to the observed bottleneck (compute or storage scale-up), and when ready capacity is admitted to traffic (readiness admission).

The thesis does not claim that this SDN-based architecture is superior to existing orchestration platforms such as Kubernetes or NFV Management and Orchestration (MANO) platforms, nor that its mechanisms generalise beyond the tested infrastructure. It claims only that these three interfaces are measurable, that within the reviewed literature they have not been previously characterised in isolation, and that measurement can establish how they affect resource orchestration and under what conditions.

1.2. Objectives

The main objective of this thesis is to study the coordination gaps that arise between independent modules with their own control loops and to contribute to more efficient resource orchestration for stateful edge services, by characterising a specific and less examined part of this problem, the coordination at the interfaces of the demand-to-capacity chain under varying demand shifts. This coordination covers the handoff delays that can arise in delivering demand evidence and in admitting ready capacity, and the capacity-action choice that determines which resource tier is relieved.

The platform keeps controller-side coordination in-process while retaining telemetry delivery as an explicit, configurable boundary. Along with explicit lifecycle hooks for scaling and admission, this arrangement exposes the selected interfaces to direct measurement. Because the components share one process and are therefore coupled, making each interface independently configurable is itself a deliberate design choice, so that the platform can vary the delivery of demand evidence, the scaling action selected, or the admission of ready capacity, and measure the effect.

1.3. Research Questions

This thesis seeks to answer the following three research questions:

- *RQ1* – How do three ways of delivering demand evidence to the controller, more specifically, event-preserving, delayed event-preserving, and latest-state, affect what the controller observes about overload, how it responds by scaling, and the quality of service users experience during a demand shift?
- *RQ2* – Under compute-bound and data-access-bound demand, how does bottleneck-aware scale-up, compared with fixed single-tier policies, affect targeted-tier relief, service recovery, and resource use?
- *RQ3* – When a newly provisioned backend becomes ready, how does the readiness-admission mechanism, direct lifecycle notification versus periodic discovery, affect when the backend actually serves requests, and does lifecycle notification become a service-quality cost when spare capacity is already low and a readiness notification is lost?

1.4. Research Methodology

The work presented in this dissertation follows the Design Science Research (DSR) methodology [11], [12] proposed in [13], and demonstrated in Figure 1.3.

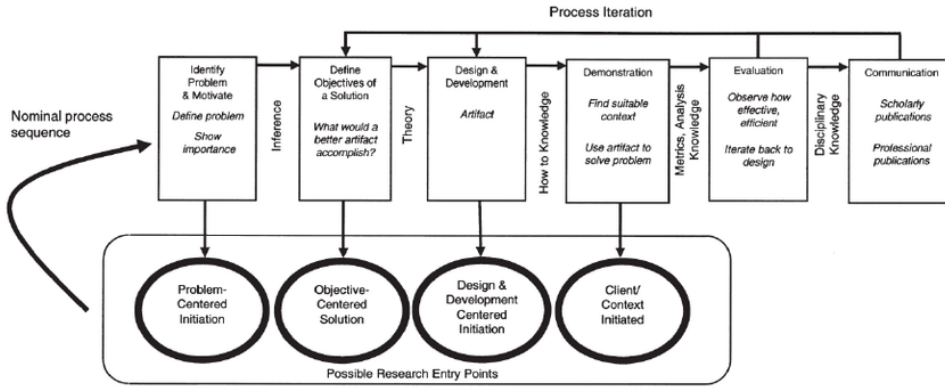


FIGURE 1.3. Design Science Research Methodology

Design science produces and evaluates purposeful artefacts that address certain real-world problems, the research contribution corresponds to the knowledge gained by designing the artefact and evaluating it against defined objectives. In Design Science Research Methodology (DSRM) the research is organised into six categories: problem identification and motivation, definition of the objectives of a solution, design and development, demonstration, evaluation, and communication.

This dissertation follows a problem-centred approach, in which the problem motivates the research and is defined as the coordination delays that accumulate between independently operating control loops in the orchestration of edge services. The problem definition is grounded in the state of the art reviewed in Chapter 2. The six activities unfolded as follows.

- (1) *Problem identification and motivation.* Edge services whose monitoring, routing and scaling modules operate as independent components accumulate handoff delays between control loops, degrading orchestration efficiency. The problem is

introduced in Section 1.1, and the review of the state of the art in Chapter 2 establishes both that these functions are typically studied in isolation and that single-process architectures with independently configurable interfaces are less commonly present in the reviewed literature, which motivates and delimits the problem.

- (2) *Objectives of a solution.* As defined in Section 1.2, the solution is an experimental platform, not a novel algorithm, whose purpose is to make the interfaces of the demand-to-capacity chain measurable, so that their impact on resource orchestration can be evaluated through measurement and better understood.
- (3) *Design and development.* The artefact is the single-process platform described in Chapters 3 and 4: two peer SDN controllers (OS-Ken/OpenFlow), one per network domain, each hosting controller-side telemetry consumption, elasticity decision-making and the double-VIP routing model, with Docker containerised service backends and a MongoDB replica set per domain serving as the storage tier, each scaled in and out through MongoDB native replication. The technology choices are justified by their utility for the platform. SDN provides a single programmable control plane through which routing behaviour can be re-programmed directly in a central manner, Docker enables fast provisioning and teardown of service backends, and MongoDB for native replica-set replication and concurrent access.
- (4) *Demonstration.* The artefact was deployed as a running system in an emulated edge environment with two simulated sites, and was driven with scheduled workload phases reproducing demand shifts in order to demonstrate that the platform operates end-to-end from demand observation through scaling action to traffic admission, with the tested interface varied across runs of each experimental campaign for evaluation.
- (5) *Evaluation.* Each research question was evaluated in a dedicated experimental campaign reported in Chapter 5. The evaluation follows a pre-registered measurement and statistical procedure. A campaign is the set of independent runs that answers one research question by varying one interface of the demand-to-capacity chain while the others are held fixed; each run executes the campaign’s workload and is treated as an experimental unit. The workloads are issued by a scheduled, open-loop workload driver, which applies the offered request schedule independently of the responses it receives, so that the offered load is preserved even when the service degrades. The primary comparisons are detailed in Section 5.1. Internal, external and repeatability validity threats are discussed in Section 6.3.
- (6) *Communication.* The communication of the problem, the artefact and the findings is accomplished through this dissertation itself and its public defence, as well as through any resulting publications.

1.5. Main Contributions of this Dissertation

This dissertation makes the following contributions:

- (1) An experimental characterisation of telemetry delivery semantics (event-preserving, delayed event-preserving, and latest-state) and of their effects on overload observability, scaling response, and transient service quality (RQ1).
- (2) An experimental characterisation of bottleneck-aware capacity action selection, more specifically how deciding between compute and storage scale-up from the observed bottleneck, compared with fixed single-tier policies, affects targeted-tier relief, service recovery, and resource use under compute-bound and data-access-bound demand (RQ2).
- (3) An experimental characterisation of readiness admission that quantifies, under direct lifecycle notification versus periodic discovery, the time from verified readiness to routable and usable capacity, and the reliability cost the faster mechanism incurs when its readiness notification is lost and spare capacity is scarce (RQ3).
- (4) A comparative synthesis of the observation, action-selection, and readiness-admission interfaces under their respective controlled workload regimes, identifying which effects were robust, workload-bound, or inconclusive.

These contributions are claimed within the evaluated two-domain testbed and workload families.

1.6. Dissertation Structure

This dissertation is organised into six chapters:

- *Chapter 1* introduces the research, presenting its context, motivation and problem statement, the objectives, the research questions that emerged, the methodology that guided the work, and the main contributions.
- *Chapter 2* reviews the background and related work. After presenting the methodology used to conduct the review, it surveys the platform technologies on which the proposed solution rests (edge platforms, SDN and OpenFlow, containers, and NoSQL document stores), then monitoring and telemetry, auto-scaling, load balancing and readiness admission on SDN, and resource orchestration on SDN, examines the default separation of monitoring, routing and scaling into independent components observed across these areas, and concludes with the research gaps that motivate this dissertation.
- *Chapter 3* presents the architecture and design of the proposed solution for edge services: the design requirements, the SDN-based architecture, the distributed elastic allocation of compute and storage resources, the monitoring and decision engine, and the control workflow.
- *Chapter 4* describes the implementation of the proposed solution: the experimental infrastructure, the implementation of the elastic allocation of compute and

storage resources, the monitoring and decision engine, the control workflow, and the validation of the implementation.

- *Chapter 5* presents the experimental results. Following the demand-to-capacity chain, the experimental campaigns address telemetry delivery semantics (RQ1), bottleneck-aware scaling action selection (RQ2), and readiness admission to traffic (RQ3), followed by a synthesis across the three interfaces.
- *Chapter 6* concludes the dissertation, restating the findings and the contributions, discussing the limitations of the work, and outlining directions for future research.

CHAPTER 2

Background and Related Work

This chapter presents the background and related work that motivated the research, selected through the search strategy described in Section 2.1. The review is separated into two parts. First, the technologies and concepts used to build the experimental platform are explained and surveyed, each introduced for what it is and why it suits the edge scenario. These are, more specifically, edge computing, SDN and OpenFlow, containerisation, and NoSQL document stores with a focus on MongoDB.

The layer that binds these technologies and concepts into a running service, known as resource orchestration, is also reviewed, alongside the three functions it covers, which are monitoring and telemetry, auto-scaling, and load balancing. Then the focus is drawn onto the interfaces between them that are part of the demand-to-capacity chain to evaluate how demand shifts are detected, which capacity action is selected when several are available, and how readiness admission is handled.

Within each area, the review examines whether and how the interfaces between these functions are considered, studied, evaluated, or mentioned. Whether the reviewed work provides evidence that the delays and losses at these handoffs cause orchestration or service-quality problems, and what ideas each field offers about them, with the aim of establishing a solid ground for the evaluation of the three interfaces of the demand-to-capacity chain.

This examination grounds the claim, introduced in Section 1.1, that these interfaces are acknowledged in the literature but have not been systematically isolated and studied. The orchestration platforms that provide the context for these functions are reviewed in Section 2.6, and the findings across all areas, together with the research gaps they define, are synthesised in Section 2.7.

2.1. Review Methodology

Searches were executed in Google Scholar and IEEE Xplore for publications from 2016 onward, with the number of citations taken into consideration. Works reached through Google Scholar were retrieved from IEEE Xplore, ACM Digital Library, Springer, and ScienceDirect. Each query resulted in roughly 30 papers evaluated for selection. Table 2.1 groups the search-term families by domain, and Table 2.2 states the selection criteria applied to the retrieved hits.

Screening and citation chaining produced the reviewed literature, 40 works organised by the interface each work informs: telemetry delivery, capacity action selection, readiness

TABLE 2.1. Search-term families by domain.

Domain	Example search terms
Monitoring and telemetry	“SDN monitoring”, “SDN telemetry”, “push vs pull telemetry”, “monitoring delay SDN”, “container monitoring edge”, “edge platform monitoring”
Auto-scaling and data	“auto scaling SDN”, “scaling trigger”, “Kubernetes HPA end-to-end latency”, “application metrics”, “edge storage survey”, “edge data placement”
Load balancing	“load balancing SDN”, “handoff delay”
Platform context	“cross-layer orchestration”, “SDN resource orchestration edge”, “edge platform survey”, “geo-distributed edge”, “stateful edge web services”

TABLE 2.2. Selection criteria.

Inclusion	Exclusion
Addresses at least one of the three interfaces in software-defined, containerised, or edge settings	Focus outside the orchestration of service capacity (e.g., physical layer, security-specific, domain applications without an orchestration contribution)
Surveys and taxonomies that frame one of the domains	Duplicates
Seminal works reached by citation chaining	Non-English or full text unavailable
Full text freely available or accessible under ISCTE’s scientific access licences	
Published from 2016 onward, with the number of citations taken into consideration, and retained on topical relevance	

admission, and platform context. Eight background sources support the platform technologies presented in Section 2.2, and are treated as context rather than as independent evidence of an RQ gap. The synthesis is recorded in Section 2.7.

2.2. Platform Technology Background

This section presents and explains the four building blocks that define the testbed evaluated in this thesis, edge computing, SDN with OpenFlow, containerisation, and MongoDB.

Each subsection states what the building block is and why it matters for the thesis scenario. How the building blocks are applied is the subject of Chapters 3 and 4.

2.2.1. Edge Computing

Edge computing is the placement of computational and storage resources between end devices and centralised cloud data centres, close to where data is produced and consumed. It complements rather than replaces the cloud, as latency-sensitive processing is shifted to the edge, while large-scale analytics and long-term storage remain in the cloud [7].

In essence, edge computing exists because centralised clouds alone cannot serve every workload efficiently, and bringing computation closer to the user lowers the latency of interactive and data-intensive services, and processing and storing data locally reduces the ingress bandwidth into the cloud [8].

Despite the advantages of edge computing, as mentioned previously in Section 1.1, the inherent geo-distribution of resources increases their management and orchestration complexity. Resource heterogeneity, scarcity, and demand varying over time are all aspects that can degrade resource orchestration efficiency, making it a concept of continuous study and research [9, 14]. Scheduling that does not follow the workload leads either to over-provisioning, where resources are held but not used, or to under-provisioning, where service quality degrades [9]. Luo et al. also identify that the joint scheduling of communication, computing, and storage resources is an open direction for the field [9].

Managing these platforms remains an open challenge, and their combination with SDN is recorded as an area with room for further investigation [15]. The service evaluated in this dissertation runs in an emulated edge environment as a stateful edge service whose capacity must follow fluctuating demand, which is the management problem the remainder of this chapter reviews.

2.2.2. SDN and OpenFlow

Software-Defined Networking separates the control plane, which decides how traffic is handled, from the data plane, which forwards packets, so that forwarding behaviour can be reprogrammed centrally from a controller instead of being configured device by device.

Baktir et al. survey how edge computing benefits from this decoupling and organise the gains into benefit areas such as high-resolution control built on a centralised view and programmatic access, arguing that SDN can act as the enabler that lowers the complexity barriers and enhances the flexibility of edge deployments [16].

The OpenFlow protocol implements the decoupling through match-action flow rules installed on the switches, and is, as reviewed by Waseem et al., the most extensively used SDN protocol in the research community [17].

In its OpenFlow form, a single controller re-programs the forwarding behaviour of the network by installing and updating match-action flow rules, so that network behaviour can follow the life cycle of the services it carries without reconfiguring the forwarding devices individually. The decoupling is illustrated in Figure 2.1.

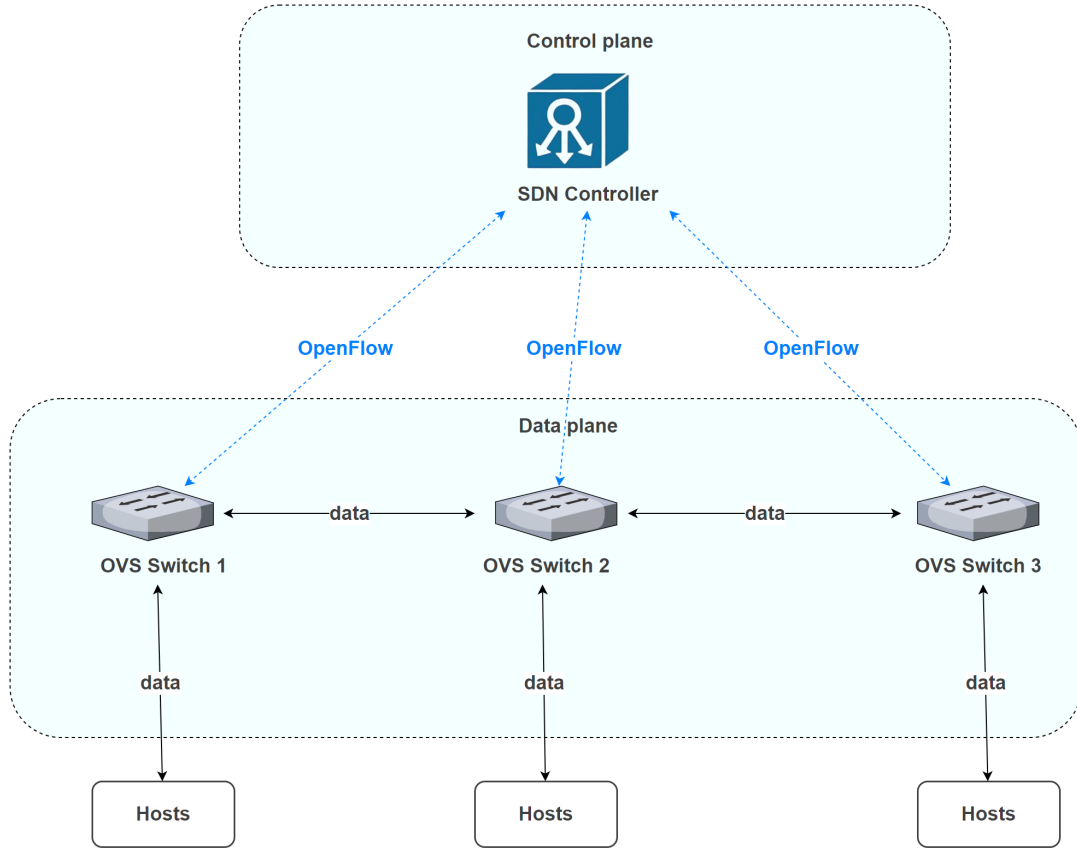


FIGURE 2.1. Software-Defined Networking architecture: the control plane is decoupled from the data plane and programs the forwarding devices through the OpenFlow protocol.

2.2.3. Containerisation and Docker

Containers package an application with its dependencies and share the host kernel, which makes them lightweight and quick to start up compared to virtual machines, properties that suit resource-constrained edge platforms. In this thesis Docker is adopted as the platform's containerisation tool because of its wide adoption for building and running containers.

2.2.4. NoSQL and MongoDB

Document-oriented NoSQL stores organise data as schema-flexible documents rather than as rows of relational tables. Because related data is embedded in a single document and the model avoids joins and fixed schemas, records can be partitioned and replicated across servers, which is how these stores scale horizontally, whereas relational systems are traditionally scaled vertically by enlarging a single server. This model suits the heterogeneous data of edge and IoT services, whose structure varies between sources and evolves over time.

MongoDB is a document store of this family. It holds data as BSON documents within collections and provides a flexible schema. In MongoDB, redundancy is organised natively around the replica set, a group of `mongod` processes that maintain the same data

set. One member is elected primary and serves the clients' writes, which it records in an operation log, the remaining members, the secondaries, replicate by applying that log asynchronously and autonomously. Members exchange heartbeats, and when the primary fails or becomes unreachable, the secondaries hold an election and promote one of their own, so the set keeps serving without operator intervention [18]. Members can also be added to and removed from a deployment while it keeps serving, which turns the replica set into a controllable unit of storage capacity rather than a fixed configuration. Figure 2.2 shows the members of a set and the replication paths between them.

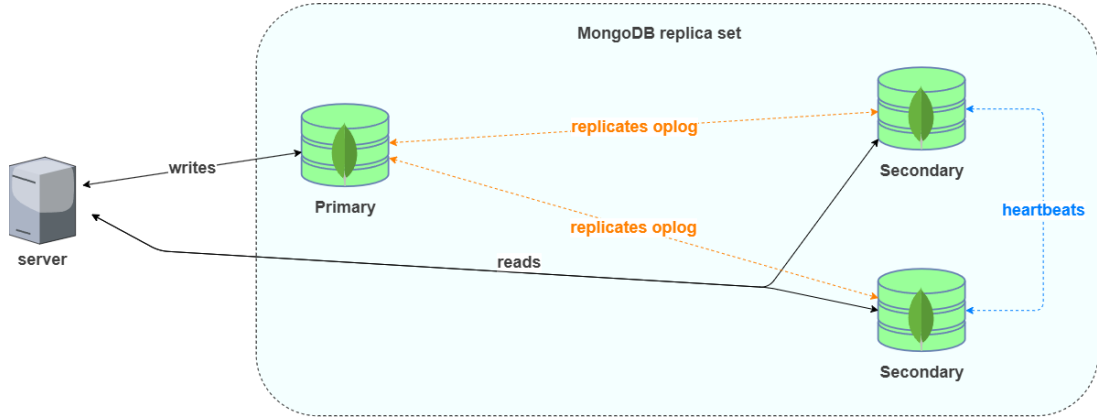


FIGURE 2.2. MongoDB replica set: a primary member serving the clients and secondary members to which the data is replicated.

This dissertation builds on that capability. The storage tier is scaled by adding and removing replica-set members while the set keeps serving, leveraging MongoDB's native replication.

2.3. Monitoring & Telemetry

Monitoring and telemetry observe the state of a running service and deliver that evidence to whoever acts on it. Monitoring is the collection of resource and load metrics, telemetry the transport that carries the collected data to the consumer, and together they form the first link of the demand-to-capacity chain. Every later decision, on scaling or on routing, can act only on the evidence this link delivers, at the time it delivers it, and with whatever intermediate observations the delivery process preserves. In edge deployments the link matters even more, because the evidence must travel from many dispersed nodes to the decision point, so any delay or gap in its delivery becomes delay or blindness in the orchestration response.

The state of the art in this area is built around a pipeline in which per-node collectors sample resource and load metrics, an aggregation layer summarises the samples, and the resulting series feed the consumers that make decisions. In orchestrators this pipeline is standardised around periodic scraping into a time-series store, as in OSM-based NFV for transparent VNF clusters, where the MANO collector polls the infrastructure into a Prometheus database that the control modules consume [3].

Delivery is conventionally pull-based, the consumer requesting the latest sample at a fixed interval [19], and this choice alone determines much of what the consumer can know. Between two collection instants the monitored state is unobserved, and what the pipeline forwards between instants determines whether the consumer receives the complete history, a complete but delayed history, or only the latest state. These different forms are defined as event-preserving delivery, delayed event-preserving delivery, and latest-state delivery. Their effect on resource orchestration is part of the subject of study in this thesis.

The reviewed evidence spans three contexts in which this link has been studied: cluster-federation monitoring, network telemetry, and edge-platform observability. In each, the review asks whether the delivery mechanism itself is varied, what downstream consequence the variation is shown to produce, and what the work leaves as a given.

Within cluster-federation monitoring, the collection interval has been shown to be a genuine variable rather than a fixed parameter. AdapPF [20] varies the Prometheus scrape interval of a geo-distributed Kubernetes federation and measures the effect on a downstream consumer. Under high load, a coarse sixty-second interval substantially degrades scheduling accuracy relative to a five-second interval, and an adaptive cadence restores it while reducing cross-cluster monitoring traffic. AdapPF is the only reviewed work that varies monitoring freshness and measures a downstream consequence, although not in the context of a stateful service.

Zhou and Yong study what to monitor rather than how it is delivered, comparing custom-metric autoscaling driven by application-level 5xx error rates against conventional CPU-based autoscaling in Kubernetes, and find that metric choice changes scaling quality under load, albeit on a small single-service testbed [21].

A second context, network telemetry, is likewise pull-based. Yaseen reviews programmable network-wide monitoring and records that SNMP polling produces “visibility gaps” between collection instants, calling for monitoring to be “tightly integrated with network control and automation” [19]. Hung et al. present an SDN controller for 5G backhaul networks that closes a telemetry-and-routing loop, with telemetry collection delegated to decentralised microservices reached through an API, showing that monitoring and control functions can be combined at one control point, although the loop omits scaling altogether [22].

At the platform level, adjacent observability work exhibits the same open dimension. Usman et al., surveying observability of distributed edge and container-based microservices, leave the definition of appropriate measurement intervals as an open research topic [23]. None of these works runs over a stateful service, so they are read here for the delivery mechanisms they expose, not as direct precedents for the platform under study.

Across the reviewed literature, therefore, the building blocks of the delivery question appear separately. AdapPF varies the interval that produces delay, and Yaseen documents the gaps that polling leaves, yet no reviewed work distinguishes delayed event-preserving

delivery from latest-state delivery, and none traces either regime through a stateful service’s scaling and traffic-admission path. This distinction is the gap this dissertation isolates and measures.

2.4. Auto Scaling

Auto-scaling adjusts the amount of allocated resources to the load that a service receives. When demand grows, additional capacity is provisioned, and when demand recedes, surplus capacity is released, so that the service keeps serving requests without holding resources it does not use. Auto-scaling strategies are mostly grouped into horizontal scaling, which changes the number of units, and vertical scaling, which changes the capacity of a single unit, with hybrid strategies combining the two [24]. Horizontal scaling suits the edge scenario, and is the direction this dissertation follows. Throughout this dissertation, adding a unit of capacity to a tier is referred to as scale-up, and removing one as scale-down.

In edge computing, a site’s capacity is bounded by the nodes deployed there, so enlarging an individual unit has a hard ceiling, whereas adjusting the number of units lets capacity grow and shrink as demand changes. When and how many resources to add or remove therefore determines both the quality of service during a demand shift and the efficiency with which scarce resources are managed.

Qu et al. classify the auto-scalers along several characteristics, among them the scaling indicator, the resource estimation method, and the timing of the action, and Taherizadeh and Stankovski present a taxonomy for edge computing whose dimensions include adaptation intervals and architectural support for multi-tier applications, noting that a scaling approach should account for different dynamic provisioning policies at each tier. Both surveys take the scaling of service instances, whether virtual machines or containers, as the unit of analysis. The main questions the field asks are when to scale and how many instances to add, which kind of capacity is provisioned, when applicable, is not among the classification dimensions [1, 25].

With SDN, the traffic side is added, but the scaled resource remains the one the service function is built from. Llorens-Carrodegas et al. couple SDN-based traffic steering with horizontal scaling of virtualised network functions, so that the SDN controllers reprogram the forwarding paths while instances of the same pre-dimensioned function pool are activated or deactivated. The decision reduces to whether and how many instances of that one function to activate.[3]. In a stateful edge service, however, the storage tier is itself a live capacity. When the pressure sits in the data rather than in the function instances, adding instances cannot relieve it.

Predictive and learning-based approaches sharpen the same decision without widening it. Toka et al. formalise the Kubernetes horizontal autoscaler’s control model to anticipate demand for edge clusters [26], Ju et al. and Cheng et al. forecast workload to adjust replica counts ahead of demand, the latter jointly with placement [27, 28], and Li et al. couple scaling and placement in a single optimisation [29]. Across these systems the

optimisation lies on the forecast and adjusting the number of compute replicas over time. The improvement pursued is a better forecast of how many instances to run, which tier holds the limiting resource does not enter the scope of study.

Scaling one resource does not always make the service better, because the pressure may sit elsewhere. Nikraves et al. quantify what that default costs in a three-tier web service. The scaling decisions taken for the business tier turn out to be insensitive to database capacity, yet reducing database capacity raises the rate of SLA violations, so service quality can be governed by the tier that the scaling decisions ignore [30]. Dogani et al. make the same observation in their survey, noting that adding resources does not necessarily raise throughput when the bottleneck lies elsewhere [31]. Among the systems covered by that survey, only a few take cross-service effects into account. Waterfall, a machine-learning auto-scaler, accounts for how scaling one microservice can shift the bottleneck towards downstream services. The mechanism remains compute-side, and no surveyed system selects a storage action.

PBScaler identifies the microservice whose degradation propagates through the invocation graph, ranks candidate bottlenecks, and optimises replica management around the identified bottleneck [32]. Bottleneck awareness therefore exists in the reviewed state of the art, but the bottleneck it locates is always a microservice and the action it takes is always replica scaling, so the selection between compute and storage capacity remains outside the action space.

The closest reviewed precedent to selecting between resource tiers is the provisioning model of Li et al., which adds both cloud compute instances and data replicas in cloud-assisted edge systems, from a forecast of future workload built over historical workload data [33]. Both actions are available, but the trigger is a forecast rather than a live observed bottleneck, and the capacity is provisioned across edge hosts and rented cloud instances, with the outcome measured in tenancy cost.

Even among the most recent surveys regarding autoscaling, El Mrabet et al. extend the classification across cloud, fog, and edge, categorising the surveyed methods by their decision mechanism, scaling strategy, and deployment layer, and the provisioned tier is not among the classification dimensions [24].

Qu et al. describe a web application as split into a front-end, application logic, and a database tier, and record that “the database tier is often considered dynamically unscalable and ignored by auto-scalers” [1, p. 6].

Overall, the literature demonstrates that the kind of capacity added under certain scenarios can have an impact, compute replicas cannot relieve pressure whose limiting resource lies in the storage tier, and database capacity can govern SLA violations while a compute-tier scaling decision remains unchanged. Capacity actions are therefore not interchangeable merely because each adds resources.

Therefore in the reviewed studies, the choice between these actions remains an architectural given, settled by forecasts or by joint placement rather than selected at runtime

from a live observed bottleneck. This dissertation isolates that selection, with storage scale-up among the actions the controller may select, and measures how the benefit divides across targeted-tier relief, service recovery, and resource use. How that decision is handed to the infrastructure is examined, with the other interfaces, in Section 2.7.

2.5. Load Balancing and Readiness Admission on SDN

Load balancing distributes incoming traffic among a set of available backends so that no single backend is overloaded while others remain underutilised. In SDN networks this distribution is programmed centrally, where the controller installs forwarding rules that steer each flow to a selected backend, and a selection policy decides which backend receives the next flow, as illustrated in Figure 2.3.

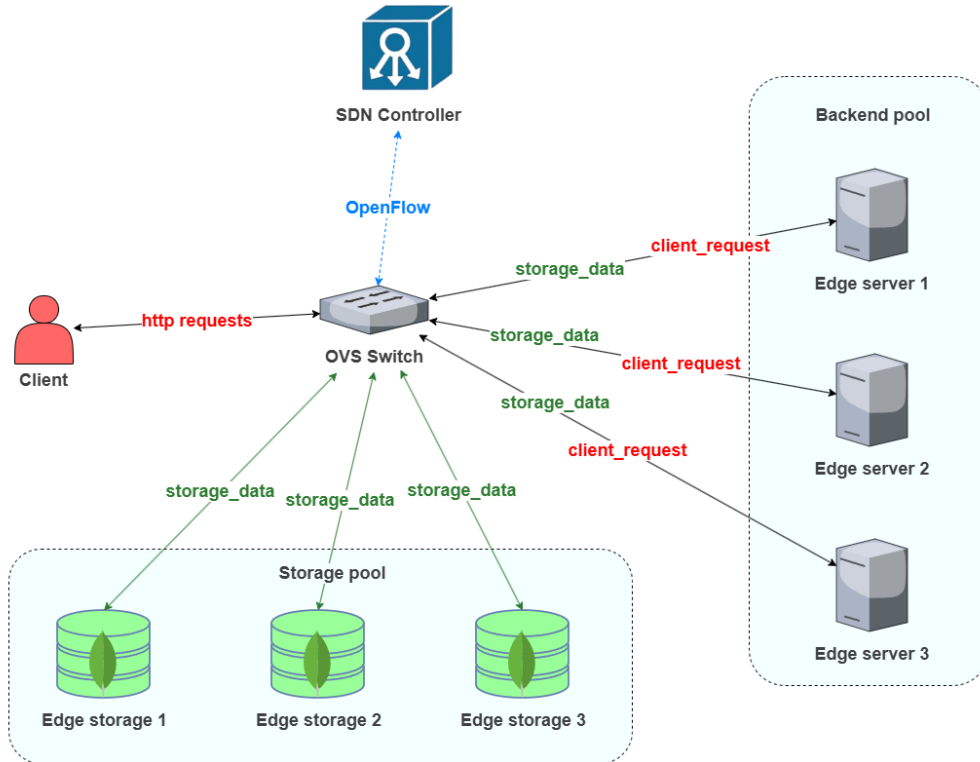


FIGURE 2.3. Load balancing in an SDN network: the controller installs forwarding rules that distribute client traffic among the available backends.

Load balancing is a common application of SDN. At a given routing decision, a load-balancing policy operates over the set of backends currently eligible to receive traffic, and the reviewed studies concentrate on optimising the distribution of load over that set, using criteria such as load, path properties, and resource utilisation [34, 35].

Kurma, a load balancer for geo-distributed storage systems, is explicitly designed to operate at a granularity of seconds so that it can “work in tandem with modern elastic controllers, thereby reducing over-provisioning and SLO violations incurred during provisioning delays” [2, p. 386]. The delay between provisioning and usable capacity is therefore a recognised cost, yet the interval in which readiness state propagates to the routing plane is not separated from the surrounding lifecycle.

Llorens-Carrodeguas et al. combine monitoring, auto-scaling, and load balancing in an SDN redirector for a transparent VNF cluster, so that load balancing operates over a pool of instances that is resized while traffic is being served. Their system changes the number of active cluster members and removes flow rules for unhealthy instances using status obtained through the MANO monitoring path [3]. It therefore demonstrates that dynamic membership is already a legitimate concern. Its evaluation, however, reports activation at the granularity of the integrated system, so the interval from verified application readiness to routing eligibility is not separated from the surrounding lifecycle.

The membership of the eligible set, however, is not fixed. Backends are added as demand rises and removed as it recedes, and a backend becomes usable capacity only once it can serve the application, its readiness state has propagated to the routing plane, and it is included in the eligible set. The interval between verified readiness and routing eligibility is therefore an admission delay, and it lies outside the scope of the selection studies reviewed here, which, as far as admission is concerned, take the eligible set as given. Indeed, Neghabi et al. record failure management as a concern that the mechanisms their survey covers have not taken into account, so failure-driven changes to the eligible set remain unaddressed [34].

The reviewed selection studies optimise the distribution of load over the eligible set, taking that set as given. This dissertation treats the set itself as the variable: it studies how the arrival of newly provisioned, ready capacity into the eligible set affects load balancing and resource management — the admission interval from verified readiness to routing eligibility, the time until the new capacity serves traffic, and how quickly that membership change reaches the load distribution and the incumbent tier. Direct lifecycle notification and periodic discovery are the two operational mechanisms of that coordination interface.

2.6. SDN-Based Orchestration Platforms

Resource orchestration is the management layer that keeps a running service within its requirements by coordinating the compute, network, and storage resources of the infrastructure on which it runs. It is the activity that the technologies and tools reviewed in Section 2.2 are assembled to serve. Edge sites supply the resources, SDN supplies the programmable connectivity, containers supply the unit of compute, and the document store holds the service state. That decision-making, and its execution, is orchestration, and at the edge it is all the more demanding because each site holds scarce capacity and the demand on it fluctuates over time. For stateful services, the capacity action can be two-sided because compute replicas serve requests, while the data they serve lives in the storage tier, so scaling can address either side. Demand for data can therefore exist without demand for compute and vice versa.

This dissertation studies the SDN-based form of this architectural layer, in which the coordination spans the SDN controller, the container runtime, and the data store, and the challenge is to keep these layers working together as demand moves across sites.

The platforms reviewed here bind these layers with different degrees of coupling, and this section examines how closely the three functions of the demand-to-capacity chain, monitoring, scaling, and routing, are held together in each, using the terms interface and coordination gap as defined in Section 1.1.

Baktir et al. organise the gains SDN brings to edge computing into benefit areas such as high-resolution control built on a centralised view and programmatic access [16]. Wang et al. survey architectures in which SDN and NFV jointly coordinate placement, traffic, and caching across the network and service layers, showing that SDN’s role at the edge extends beyond forwarding into orchestration [36]. Sofia et al. argue for context-aware cross-layer orchestration of containerised edge-cloud applications serving next-generation IoT services, treating contextual information such as location and data freshness as part of the orchestration input [37]. These works document architectures or proofs of concept rather than the coordination inside a running system, and the interfaces between the functions fall outside the scope of all three.

Peng et al. exercise SDN-based resource management in vehicular multi-access edge computing as task migration and radio slicing, treating telemetry as available input rather than as an interface to be studied, and without container orchestration or tier scaling [38].

Some of these platforms bring the functions together at the SDN control point itself. Llorens-Carrodegas et al. combine monitoring, auto-scaling, and load balancing in a single SDN redirector for a transparent VNF cluster, although metric collection and the lifecycle operations that feed and execute the decision remain external [3]; Hung et al. present an SDN controller for 5G backhaul that closes a telemetry-and-routing loop, with telemetry collection delegated to decentralised microservices reached through an API, omitting scaling altogether [22]. The first study binds all three functions at one control point and the second binds two of the three, so combining the functions at an SDN control point has precedent in the literature and is leveraged in this dissertation. Neither system, however, reports the delay within its coordination.

At the geo-distributed scale, three systems coordinate the same functions. OneEdge keeps authoritative state at each edge site, reconciles it with a central aggregate through two-phase commits, and uses ZeroMQ (ZMQ) for communication among its distributed components. Its evaluation reports end-to-end deployment latency with a coarse breakdown into scheduling delay, WAN overhead, and container creation, but not the cost of each control-plane interaction [39]. McK8s deploys four controllers on a management cluster as Kubernetes operators over custom resources, connects member clusters through a Cilium mesh, and measures inter-cluster latencies with Serf, treating those latencies as input to placement rather than as an effect to study [40]. Kurma instances exchange load, latency, and SLO-violation measurements among datacenters and redirect requests at second granularity while elastic controllers provision capacity, naming the stages of the provisioning delay, detection, acquisition, and warm-up, before treating them as one combined quantity [2]. The coordination is built into each system, but the delay it carries

is not isolated. None of their evaluations attributes the time taken by the state updates, exchanges, and redirects along those communication paths to the individual handoffs between observing demand, deciding, and admitting capacity.

The related background is therefore organised in two modes. In the dominant mode, monitoring, scaling, and routing are implemented as separate components, each running its own control loop, so the evidence and commands that tie them together must cross component boundaries. In a second, recurring mode, the same functions are kept together at an SDN-based control point or coordinated through shared state. What neither mode provides, however, is a decomposition of the control process into its interfaces. Prior systems measure complete control processes or coordinate them by construction. They do not isolate the contribution of telemetry delivery, capacity-action selection, or readiness admission to the demand-to-capacity path. This interface-level contribution is precisely the quantity this dissertation highlights.

SDN-based resource orchestration is therefore an established and feasible arrangement, and the coordination it rests on is a recorded open issue. Nain et al. record resource optimisation in SDN-based edge environments among the continuing challenges of the field [15]. The handoff delay studied in this dissertation is one measurable dimension of that coordination, and because the functions meet at boundaries that an SDN-based control point can expose as explicit, configurable interfaces, such a platform is a suitable experimental basis for evaluating the handoff delays between them. The platform built in this dissertation follows that arrangement and exposes telemetry delivery, capacity-action selection, and readiness admission as independently configurable interfaces, as presented in Chapter 3.

2.7. Synthesis and Research Gaps

The review has examined the functions of resource orchestration. Where the reviewed works address the delay between the functions, it is treated at three levels. The delay is first acknowledged as a real cost. Kurma’s seconds-granularity operation aims to reduce the over-provisioning and SLO violations incurred during provisioning delays [2], and Yaseen records the visibility gaps that pull-based polling leaves and calls for monitoring tightly integrated with control [19]. Second, where the delay is evaluated, it is evaluated only in composite. Llorens-Carrodegua et al. report activation at the granularity of the integrated system, OneEdge reports end-to-end deployment latency breaking it down into scheduling, network, and container-creation components, and Kurma presents the provisioning delay as one combined quantity, none attributing the measured time to an individual handoff [2, 3, 39]. Third, the delay is never materialised as an experimental variable. The interfaces of the demand-to-capacity chain are acknowledged and, in places, combined, but no reviewed work traces the effect of an individual interface through a stateful service. The closest reviewed precedent is AdapPF, which varies monitoring freshness and measures a downstream consequence, but only for a single evaluated consumer, a

scheduler [20]. The separation that produces these delays is not an artefact of one platform family but the shared default of the areas reviewed. What varies across them is only how far each acknowledges the boundaries it imposes. The placement literature, as surveyed by Tabatabaee Malazi et al., for instance, names the interval between an instance’s arrival and its readiness to serve as launch time, but the step from readiness to traffic admission lies outside its scope [41].

Table 2.3 condenses these findings into the three interfaces and the question each leaves open.

TABLE 2.3. The three interfaces of the demand-to-capacity chain and the questions they leave open.

Interface	Open question
Telemetry delivery to decision (RQ1)	Does a controller behave differently when demand evidence arrives late but complete, versus when intermediate observations are absent?
Decision to capacity action (RQ2)	When compute and storage actions are both available, how much does selecting the action matched to an observed compute-bound or data-access-bound bottleneck change targeted-tier relief, service recovery, and resource use relative to fixed single-tier policies, and where do the effects concentrate?
Ready backend to traffic (RQ3)	How does readiness-admission coordination affect the delay from verified readiness to usable capacity and its transition-window service consequence?

The research questions of Section 1.3 are the experimental form of Table 2.3. This dissertation does not argue that the coordination gap is universal, only that, within the reviewed literature, the interfaces that compose it have not been isolated as independent variables.

The evaluation follows the same per-interface discipline, and the demand-to-capacity chain is treated as a sequence of separately measured interfaces, whose aggregate cost this dissertation deliberately does not claim.

Read together, the reviewed works provide evidence of the topics studied in this thesis: the freshness of demand evidence and its effect on the link between monitoring and decision, the cost of scaling the wrong resource when the bottleneck lies elsewhere, and the interval between provisioning and usable capacity. They also show that coordinating these functions at an SDN control point is an established approach.

The questions also sit between the surveyed fields rather than inside any of them. Each area’s abstraction serves its own question. Chapter 3 presents the platform designed to make them askable and measurable.

[This page is intentionally left blank.]

CHAPTER 3

Architecture and Design of the Proposed Solution for Edge Services

This chapter presents the architecture and design of the proposed cross-layer SDN orchestration platform for stateful edge services. The design provides an experimental environment in which traffic steering, controller-side telemetry consumption, and elasticity coordination operate within each domain's controller process. The delivery of aggregated telemetry, the selection of the capacity action, and the lifecycle transitions from capacity action to traffic admission remain explicit observation points. The evaluation of the mechanisms described in this chapter is presented in Chapter 5.

Section 3.1 defines the requirements that guided the proposed solution, and Section 3.2 presents the two-domain architecture and its communication paths (Figure 3.1). Section 3.3 describes the distributed elastic allocation of compute and storage resources, Section 3.4 the monitoring and decision engine, and Section 3.5 the control workflow. Together, these sections realise the design requirements: cross-layer coordination (DR1) and fast-path isolation (DR5) through the single-process architecture and its execution contexts, independent compute and storage elasticity (DR3) and lifecycle-safe admission (DR4) through the elastic allocation, separability of the control interfaces (DR2) through the configurable delivery, selection, and admission designs, and end-to-end observability (DR6) through the instrumented chain.

3.1. Design Requirements

The platform was designed to support stateful edge services whose demand and resource requirements vary over time. The requirements below follow from the coordination problem and the research gaps established in Chapter 2, and guide the design presented in the remainder of the chapter.

- DR1: Cross-layer coordination.* Within each domain, traffic steering, telemetry consumption, topology awareness, and resource lifecycle management must share controller state within the same process. Aggregated telemetry must reach the controller through a structured, configurable delivery interface. Peer controllers must share the topology state needed for cross-domain awareness.
- DR2: Separability of control interfaces.* The delivery of demand evidence, the selection of a capacity action, and the admission of ready capacity to traffic must remain independently configurable.

- DR3: Independent compute and storage elasticity.* The platform must distinguish an increase or decrease in compute capacity from an increase or decrease in storage capacity, allowing each resource tier to be managed separately.
- DR4: Lifecycle-safe admission.* Provisioned resources must not be regarded as usable capacity before they are admitted to the appropriate routing path, and where a readiness signal is configured, it must be satisfied before admission.
- DR5: Fast-path isolation.* Packet handling must remain separate from slow infrastructure operations, such as container creation, replica preparation, and resource removal.
- DR6: End-to-end observability.* The platform must record the path from demand observation to the first successful request served by newly available capacity.

3.2. Architecture

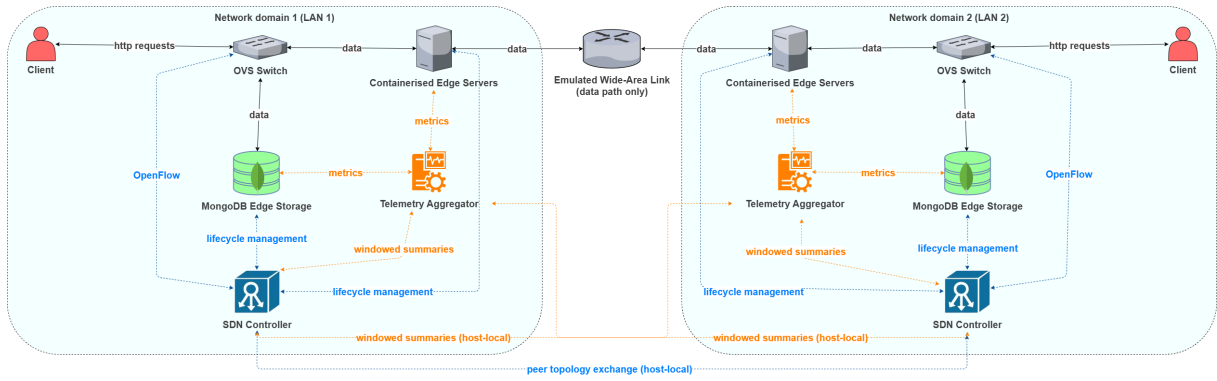


FIGURE 3.1. Architecture of the proposed platform: two network domains whose clients, data planes, edge servers, storage tiers, and aggregators are coordinated by peer SDN controllers, with host-local control paths and an emulated wide-area data link between the domains.

3.2.1. Two-Domain Organisation

The platform is organised as two network domains connected by an emulated wide-area data link. The two domains represent the two geo-distributed sites of the deployment scenario evaluated in this dissertation. Each domain contains client workloads, an Open vSwitch (OVS) data plane, a pool of containerised edge servers, a MongoDB replica-set storage tier, a telemetry aggregator, and an SDN controller. The peer controllers exchange topology information, and each consumes the telemetry summaries published by the aggregators of both domains. Each controller runs traffic steering, telemetry consumption, and elasticity coordination in the same process for its own domain.

The emulated wide-area link represents cross-domain data traffic. In contrast, telemetry delivery and controller-to-controller topology exchange use host-local control channels in the experimental environment. This distinction separates data-path delay from control-path communication.

3.2.2. Virtual IP Data Paths

Clients access edge services through a per-domain Virtual IP (VIP). The virtual service addresses are `VIP_SERVER_N1` and `VIP_SERVER_N2`, collectively referred to as `VIP_SERVER`. Edge servers route storage reads through the virtual data addresses `VIP_DATA_N1` and `VIP_DATA_N2`, collectively referred to as `VIP_DATA`, and issue storage writes directly to the replica-set primary. This separation represents service routing and data access as distinct routing surfaces.

When a flow addressed to a virtual IP is first observed, the controller selects an eligible backend and configures the OVS data plane to forward the flow. Subsequent matching packets are handled in the data plane without requiring repeated controller intervention. Service backend selection can include eligible backends in either domain when the topology permits. Cross-domain service placement is a platform capability held constant in the evaluation, while cross-region storage placement is disabled.

Backend and storage-member selection are cost-based. The controller computes a Weighted Sum Model (WSM) cost from each eligible backend's cached resource metrics (CPU utilisation, memory usage, request count, and hop distance) and installs forwarding rules for the lowest-cost one. For storage reads observed on the virtual data path, the same mechanism selects a replica-set member, using the corresponding storage metrics (CPU utilisation, memory usage, connections, replication lag, and hop distance). Figure 3.2 walks through the resulting sequence.

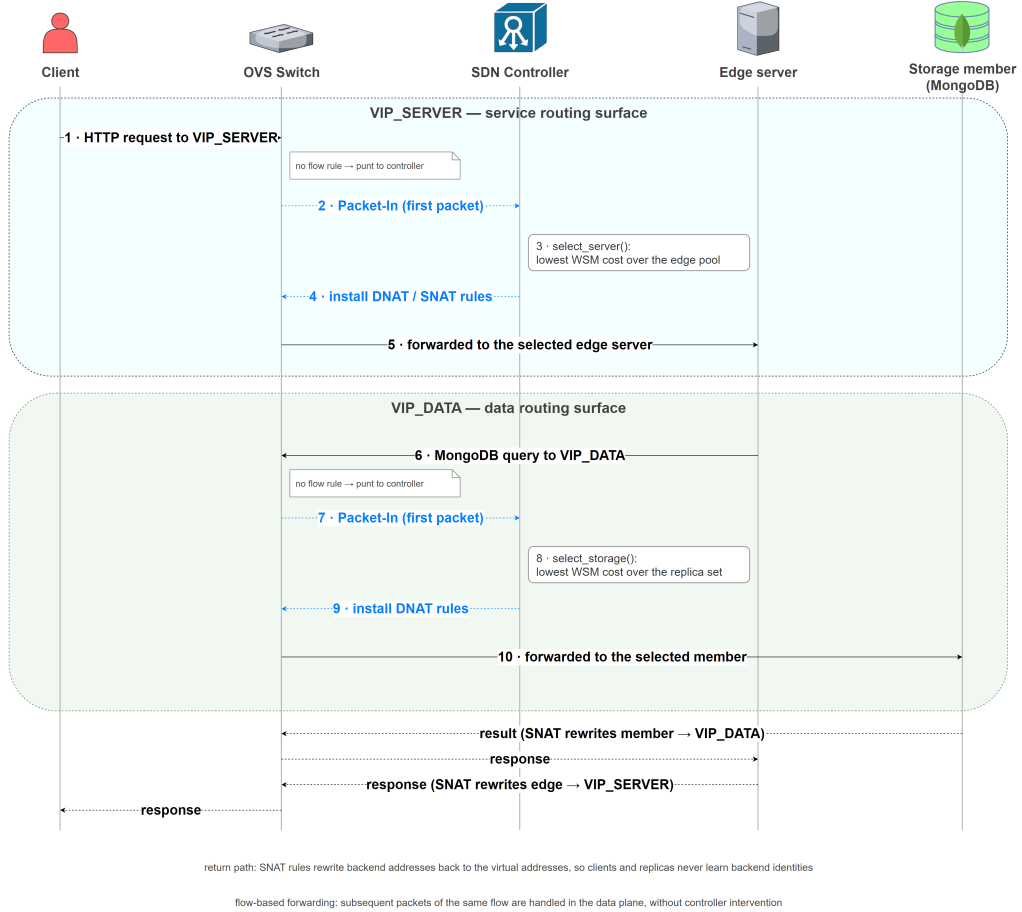


FIGURE 3.2. Double-VIP routing sequence: a client request addressed to VIP_SERVER triggers backend selection in the controller, and the edge server’s storage reads through VIP_DATA trigger the selection of a replica-set member.

3.2.3. Control and Observation Paths

Edge servers and storage components emit resource and request observations to their local telemetry aggregator through ZMQ. Each aggregator forms windowed summaries that are consumed by the controller. These summaries inform both backend selection and elasticity decisions, while topology exchange maintains awareness of peer-domain resources. The aggregation-to-controller link is the configurable telemetry-delivery interface, shown in Figure 3.3.

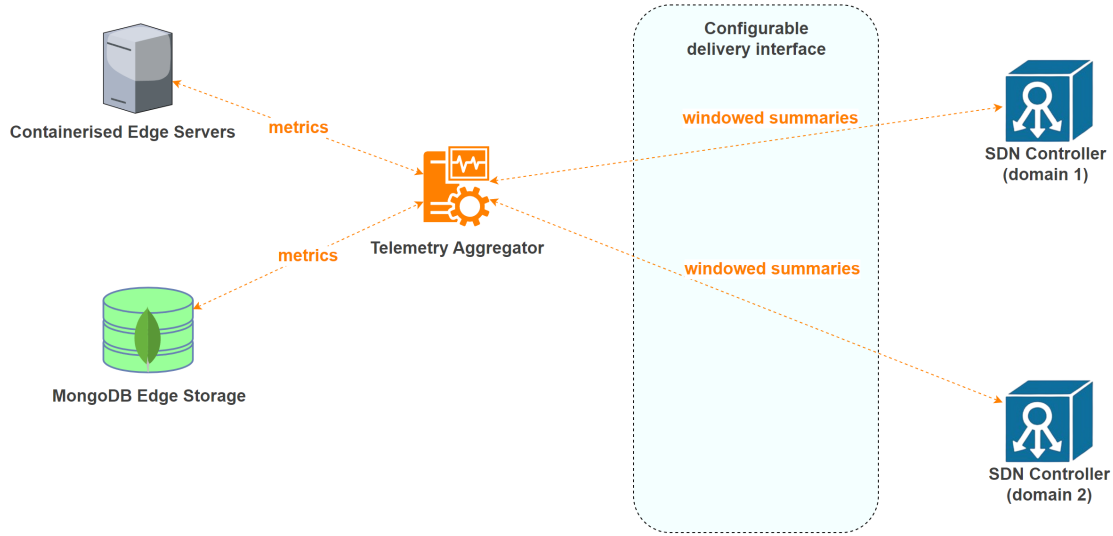


FIGURE 3.3. Control and observation paths: edge servers and storage members publish observations to the per-domain telemetry aggregator, whose windowed summaries reach both peer controllers through a configurable delivery interface.

The architecture therefore separates the request data path, the telemetry observation path, and the slow infrastructure-management path while retaining their coordination within each domain’s controller process. The concrete implementation of these paths is described in Chapter 4.

The deployment evaluated in this dissertation runs the two domains on a single host, with an emulated wide-area link. The evaluation targets the coordination interfaces between the control functions rather than wide-area transport performance, and the emulated link provides the controlled, repeatable conditions that these interface measurements require. No performance claim is therefore made for a real geo-distributed wide-area deployment. The testbed is described in Section 5.1.

3.3. Distributed Elastic Allocation of Resources

The platform scales capacity along two dimensions: edge-service replicas (also referred to as backends in the routing context), which serve requests, and storage members, which hold the service state. Each dimension has its own observation signals, its own action, and its own admission and release procedure, so that pressure on the compute tier and pressure on the storage tier can be addressed without conflating the two. The two dimensions are summarised in Figure 3.4 and described in turn.

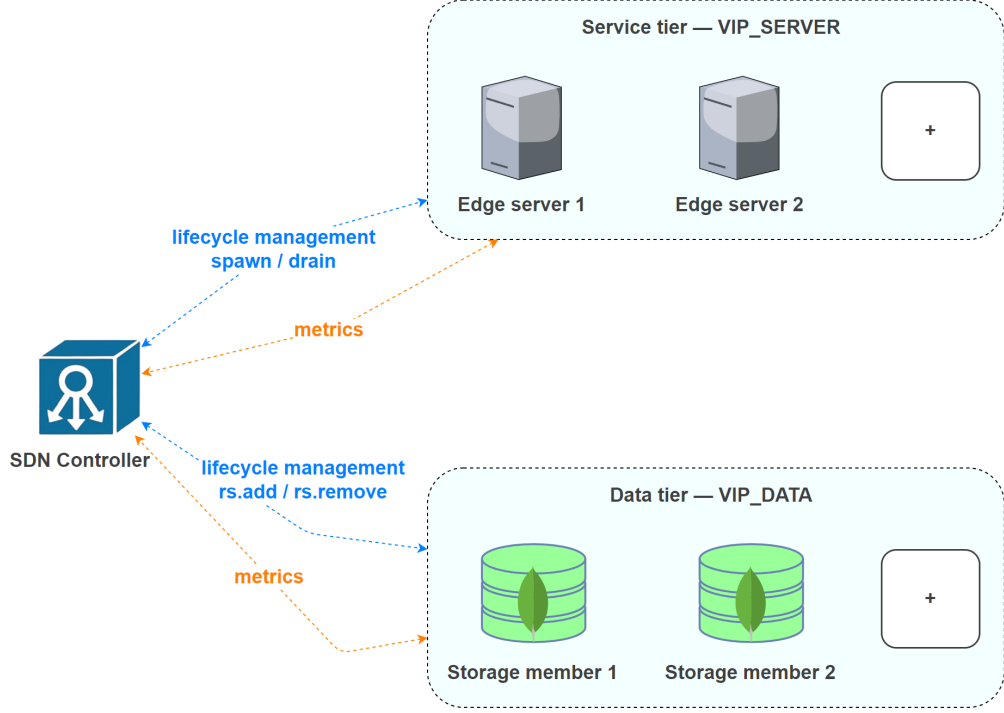


FIGURE 3.4. Elastic allocation of resources: each tier is scaled on its own observation signals, with compute replicas provisioned and drained and storage members added and removed through replica-set membership, both actions issued by the controller over the infrastructure-management path.

3.3.1. Scaling Policy

The policy is applied per resource tier, with tier-specific signals and parameters. Each tier is observed through two signals: a utilisation signal, which in this platform is the CPU utilisation of the tier’s units, and a latency signal. For each telemetry window w , let $c(w)$ denote the window’s utilisation and $t(w)$ its typical latency (a per-campaign window statistic). Each signal is first mapped onto the unit interval through two calibration parameters, a floor and a span. A value at or below its floor contributes nothing, a value at or above the floor plus its span contributes the maximum, and intermediate values contribute proportionally. The aim is to achieve a system where the floor separates normal operation from pressure, and the span sets how far the signal must rise before its contribution saturates. Because both signals are mapped onto this bounded scale, they contribute on common terms despite being measured in different units, and a single extreme window cannot dominate the score. The two normalised components are combined with weights w_{util} and w_{lat} into the degradation score

$$\begin{aligned}
 s(w) &= w_{\text{util}} \hat{c}(w) + w_{\text{lat}} \hat{t}(w), \\
 \hat{c}(w) &= \min \left(1, \max \left(0, \frac{c(w) - c_0}{c_{\Delta}} \right) \right), \\
 \hat{t}(w) &= \min \left(1, \max \left(0, \frac{t(w) - t_0}{t_{\Delta}} \right) \right),
 \end{aligned} \tag{3.1}$$

with the per-tier weights configured to sum to one, so the score lies between zero and one, where:

- $c(w)$ – CPU utilisation of the scored tier observed in window w ;
- $t(w)$ – typical latency of the scored tier observed in window w ;
- c_0, t_0 – signal floors: at or below these values a signal is considered normal for the tier;
- c_Δ, t_Δ – signal spans: the rise above the floor at which a signal’s contribution saturates;
- $\hat{c}(w), \hat{t}(w)$ – normalised components of window w , each in $[0, 1]$;
- $w_{\text{util}}, w_{\text{lat}}$ – configurable weights of the utilisation and latency components.

The floors, spans, weights, and threshold parameters are configured independently for each tier, so the policy can be calibrated to each tier without changing the structure of the score. For the compute tier the signals are CPU utilisation and request-processing latency, while for the storage tier they are storage-member CPU utilisation and data-access latency.

A window is considered degraded when its score reaches an adaptive threshold, and a scale-up is requested only when degradation is sustained across a required number of recent windows. Let $\mathcal{W}_n$ be the set of the last W windows in which the tier was eligible to act, and R the required number of degraded windows; the scale-up condition is

$$|\{w \in \mathcal{W}_n : s(w) \geq \theta(k)\}| \geq R, \quad \theta(k) = \min(\theta_{\text{base}} + k \Delta\theta + \beta_{\text{peer}}, \theta_{\text{max}}), \quad (3.2)$$

where:

- $\mathcal{W}_n$ – sliding set of the most recent W windows in which the tier was eligible to act;
- W – number of eligible windows in the scale-up window;
- R – minimum number of degraded windows required to request a scale-up;
- k – number of dynamic units currently in the tier’s pool (units provisioned by the controller beyond the initial pool);
- $\theta(k)$ – adaptive degradation threshold with k dynamic units in the pool;
- θ_{base} – base threshold;
- $\Delta\theta$ – threshold increment per additional dynamic unit, applied linearly for the compute tier; for the storage tier the increment diminishes with each additional unit, down to a configured minimum;
- θ_{max} – upper bound of the adaptive threshold;
- β_{peer} – peer relief added to the compute-tier threshold while the peer domain is healthy enough to absorb spillover traffic (zero otherwise).

The threshold $\theta(k)$ adapts upward as capacity grows, so each additional unit faces a higher bar, bounded by θ_{max} , and β_{peer} raises the compute-tier threshold slightly while the peer domain is healthy enough to absorb spillover traffic. A scale-up cooldown after each committed scale-up and a scale-down cooldown keyed on the last scale-up fire keep the pace of actions bounded, together with a per-tier cap on the number of units. The window advances only in windows where the tier was eligible to act (not blocked by an ongoing infrastructure operation, not at its cap, and not within a scale-up cooldown), and

it is cleared when a scale-up fires. A committed scale-up also clears the tier’s scale-down window, so a recent scale-up is not immediately reversed.

A window is declared idle when both signals fall below their idle thresholds,

$$c(w) < \tau_c \quad \text{and} \quad t(w) < \tau_t, \quad (3.3)$$

where:

- τ_c, τ_t – idle thresholds of the utilisation and latency signals.

A scale-down is requested only when idleness is sustained across $R_\downarrow$ of the last $W_\downarrow$ windows, where $R_\downarrow$ is the required number of idle windows and $W_\downarrow$ the length of the scale-down window. This window is longer than the scale-up one, which prevents a transient data-bound stall from being misread as idle capacity. Windows whose latency exceeds a ceiling are treated as indeterminate and do not count toward idleness.

The policy evaluates each tier per telemetry window, in a fixed order:

- (1) the window is skipped when the tier is ineligible: an infrastructure operation is in progress, the tier is at its cap, or a scale-up cooldown is active;
- (2) the degradation score $s(w)$ is computed from the two normalised signals (3.1);
- (3) if at least R of the last W eligible windows reach the adaptive threshold $\theta(k)$ (3.2), the tier requests a scale-up and the window history is cleared;
- (4) otherwise, if both signals fall below their idle thresholds (3.3), idleness is tracked, and the tier requests a scale-down once idleness spans $R_\downarrow$ of the last $W_\downarrow$ windows.

The concrete parameter values used in the evaluation are reported in Section 5.1.

3.3.2. Compute

Compute capacity is a per-domain pool of containerised edge servers, each capable of serving any request addressed to the domain’s virtual service address. Storage reads performed by a service replica are routed through the domain’s virtual data address, or served from a Tier 1 copy where one is held (the data-placement tiers are defined in Section 3.3.3), and its writes are issued directly to the replica-set primary. The replica therefore holds no knowledge of the replica-set membership, since member selection for reads is left to the routing path. Compute scale-up applies the policy defined in Section 3.3.1, with CPU utilisation and request-processing latency as signals.

A newly provisioned service replica is recorded through an explicit registration step and it becomes eligible for backend selection only once it is visible in the routing pool used by the selector. How readiness reaches that pool is itself a configurable design dimension as readiness may be propagated by a direct lifecycle notification from the replica, or discovered by the controller through periodic probes. This readiness-admission mechanism is the interface examined for RQ3.

Compute scale-down removes the least disruptive eligible replica, applying the idle condition defined in Section 3.3.1. Removal proceeds in two phases: the replica is first withdrawn from the routing pool and signalled to finish its in-flight requests, and only

after it drains is its network attachment removed. A pending drain can be cancelled if load rebounds before the replica exits, favouring fast recovery over aggressive reclamation.

3.3.3. Storage

The replica set is the unit in which storage capacity is scaled. The storage tier of each domain runs as a MongoDB replica set whose primary serves the clients' writes and whose secondaries replicate the data and absorb the remainder of the traffic. Reads may therefore be served by any replica-set member, and the consistency contract is primary writes with eventually consistent secondary reads. The evaluation does not rely on read-after-write semantics. Storage scale-up is a membership change. The controller provisions a new storage member, which joins the set as a non-voting secondary, and admits it to the data path only once the member has reached the state at which it can serve reads, so that traffic is not directed to a member that cannot yet serve it. In the evaluated configuration, preparation is decoupled from the reaction: a readied member is held in reserve outside the routing pool, and the storage scale-up is the activation of this prepared reserve, which removes the initial readiness wait from the reaction path. A new reserve is prepared immediately afterwards.

Storage scale-down removes an idle member while the remaining members keep serving. Only members added by the platform are eligible for removal, so the set's original membership is left undisturbed, and the decision applies the idle condition defined in Section 3.3.1 to the storage tier's idle signals (its data-access latency and, in the configurations that enable it, its storage utilisation).

Beyond the membership change, the platform defines three data-placement tiers, named here for completeness: Tier 0 provides direct remote reads over the virtual data path, Tier 1 holds a partial hot copy in the consumer domain, and Tier 2 is the replica-set extension described above. Tier 0 and Tier 1 are platform capabilities rather than evaluated treatments, as in the experiments, the storage action is limited to activating the prepared reserve or removing a member of the domain's own replica set.

3.4. Monitoring and Decision Engine

Observation begins at the producers. Each service replica emits a per-request record of its resource state and request experience, and each storage member's companion process emits replication and resource-state samples as the member reports activity, with an optional periodic heartbeat. A per-domain aggregator collects these records and publishes a windowed summary at a fixed cadence, so that time is observed as a regular sequence of windows and every window, including idle ones, is evaluated and recorded. The summary contains the utilisation and latency signals that the scaling policy consumes, together with the per-backend metrics used by routing.

The delivery of these summaries to the controller is an explicit, configurable interface. The default delivery pushes each completed window as it is published. Alternative delivery modes supply the same windowed evidence in other forms, such as periodic retrieval of

only the most recent window (latest-state delivery) or ordered replay of a retained window history (event-preserving delivery, immediate or delayed), so that the effect of delayed or incomplete demand evidence can be studied while the aggregation, the workload, and the decision logic remain unchanged.

Inside the controller, the summaries feed two consumers. The first is the routing state. Per-backend metrics update the state that backend selection uses. The second is the decision engine. Per-domain degradation scores, computed per resource tier from bounded utilisation and typical request latency, are evaluated against adaptive thresholds over sliding windows. When a tier's degradation condition is sustained, the decision engine emits a typed, prioritised request for a capacity action. Requests that add or preserve capacity take precedence over requests that remove capacity. Among the additions, a storage scale-up precedes a compute scale-up, and among the removals, a storage scale-down precedes a compute scale-down. The type of a request identifies the resource tier it concerns (compute or storage) and the action requested (scale-up or scale-down), and the evidence behind each request is recorded in the decision log for the window that produced it. Scale-down requests arise from separate idle conditions. Figure 3.5 details the per-window evaluation, from the eligibility check to the request emissions.

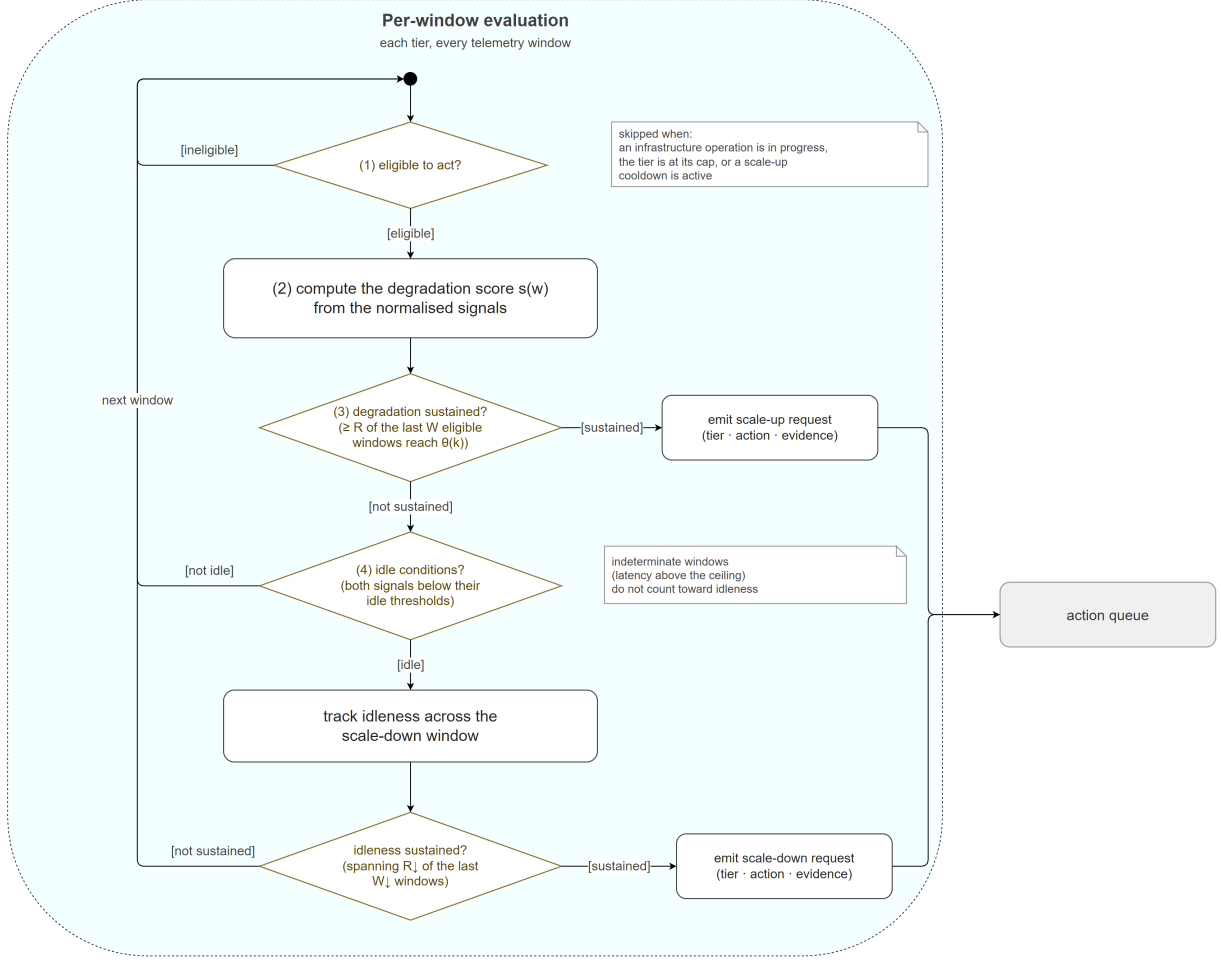


FIGURE 3.5. The per-window evaluation, in the fixed order of Section 3.3.1: (1) the eligibility check, where the window is skipped when the tier is ineligible; (2) the degradation score $s(w)$, computed from the normalised signals; (3) the scale-up condition: a scale-up request is emitted and the window history is cleared when at least R of the last W eligible windows reach the adaptive threshold $\theta(k)$; and (4) the idle condition, tested otherwise: idleness is tracked, excluding indeterminate windows, and a scale-down request is emitted once it spans $R_{\downarrow}$ of the last $W_{\downarrow}$ windows. Every remaining branch returns to the next telemetry window.

The decision engine may also be configured to commit to a single action per window when several tiers signal simultaneously, which is the selection interface of the demand-to-capacity chain. In the bottleneck-aware configuration, that selection classifies the episode as compute-bound or data-access-bound and commits the action matched to the identified bottleneck, recording the selected and rejected actions as evidence. Lifecycle signals, such as the completion of a drain or the readiness of a storage member, reach the same decision path as control events, so that admission and release follow the same disciplined flow as the actions themselves.

3.5. Control Workflow

The control workflow separates three execution contexts. They share one process but do not block one another. The packet context handles flow decisions and does not wait on slow work. The observation context evaluates telemetry windows and emits action requests. The action context performs infrastructure mutations such as provisioning, drain, and removal.

The workflow traverses the demand-to-capacity chain introduced in Section 1.1. A change in demand alters the request experience reported by the service replicas, and the window that contains the change is published and delivered. The decision engine evaluates the delivered window. If the tier condition is sustained, the engine emits an action request, which the action context executes. The provisioned unit reaches its readiness state and is admitted to the corresponding routing pool, which for the storage tier is the activation of the prepared reserve. New flows can then be directed to it, while flows already installed on other members remain where they were placed. The chain completes when the new unit serves its first successful request. Each step of this chain is instrumented, so the timeline from demand observation to usable capacity is recorded end to end (Figure 3.6), with registration, routing-pool inclusion, and the first successful request recorded as distinct events. When readiness admission is enabled, a service replica that does not reach readiness within the configured bound is abandoned and torn down instead of being admitted. Without readiness admission, service replicas are registered as soon as they are provisioned and no readiness bound applies, so a replica that never becomes usable is reclaimed only by the ordinary lifecycle checks.

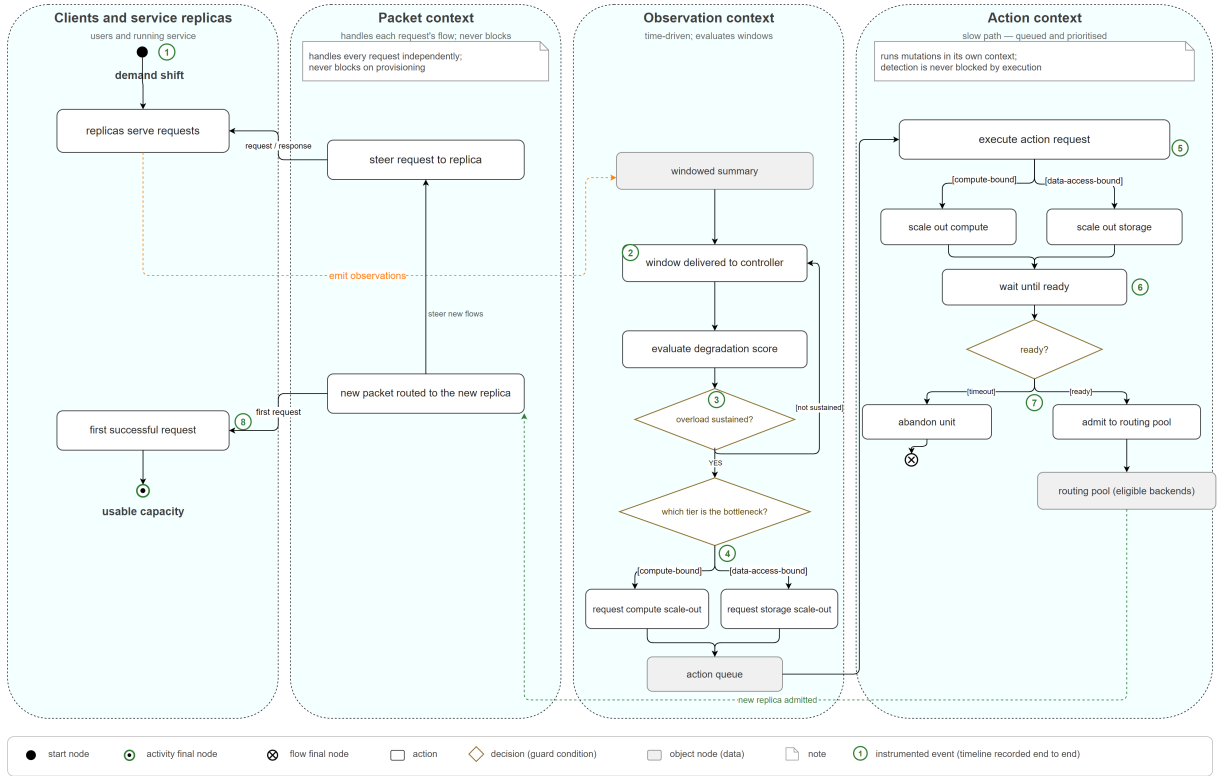


FIGURE 3.6. The control workflow, drawn as a UML diagram. The demand-to-capacity chain runs across three contexts in the same process: the packet context, the observation context, and the action context. The contexts do not block one another and they exchange state through shared structures (the telemetry state, the action queue, the lifecycle state, and the routing pools). The numbered stages match the chain in Figure 1.2 and show the provisioning direction. Capacity removal follows the same steps in reverse, starting with the drain. The aggregator is omitted between the observations sent by the replicas and the windowed summary delivered to the controller.

State crosses these contexts through shared structures rather than through direct calls. The observation context writes the telemetry state and the action requests, and the action context owns the infrastructure mutations and records the lifecycle state of each in-flight operation. The routing pools are updated with the lifecycle transitions: a unit enters them when it is admitted and leaves them on removal, while a readied member held in reserve remains outside them until activation. The packet context reads both on each flow decision. Because the packet context is not blocked by the action context, a long-running mutation cannot stall request steering. Because the action requests are queued and prioritised, detection is not blocked by execution.

Every request that adds or preserves capacity precedes every request that removes capacity. The cancellation of a pending drain precedes its execution. This execution order is distinct from the selection of the action itself, which the observation context decides before the request is queued.

Across domains, the peer controllers exchange topology state so that each controller is aware of the peer domain's resources, while acting on its own domain's demand. The concrete realisation of this design, the technologies that implement each context, and the validation of the resulting platform are the subject of Chapter 4. The experimental characterisation of the three interfaces is presented in Chapter 5.

CHAPTER 4

Implementation of Proposed Solution

4.1. Experimental infrastructure

4.2. Distributed Elastic Allocation of Resources

4.2.1. Compute

4.2.2. Storage

4.3. Monitoring and Decision Engine

4.4. Control Workflow

4.5. Implementation Validation

[This page is intentionally left blank.]

CHAPTER 5

Experimental Results

5.1. Experimental Setup

This section defines the conditions under which the three experimental campaigns of this chapter were run: the testbed and its instrumentation, the demand model and the workload of each campaign, the measured outcomes, and the design of each campaign.

5.1.1. Testbed and Instrumentation

The evaluation was performed on the platform of Chapters 3 and 4, deployed as a two-domain testbed on a single host. Each domain runs its own data plane, edge-service replicas, storage tier, telemetry aggregator, and SDN controller, and the two domains are joined by an emulated wide-area link. Both domains are driven by the same scheduled, open-loop workload driver, described in Section 4.1: the driver applies the workload’s request schedule regardless of the responses it receives, so that the demand offered to the service is independent of the capacity available to serve it.

Table 5.1 summarises the configuration common to the three campaigns and the per-campaign resource caps and quotas are listed in Tables 5.2 to 5.4.

TABLE 5.1. Evaluation testbed configuration common to the three campaigns.

Component	Configuration
Host	both domains on a single cloud virtual machine; all components run as CPU-capped containers
Wide-area link	emulated inter-domain link with a fixed symmetric round-trip latency of 185 ms (tc-netem)
Data plane Controllers	one Open vSwitch per domain, programmed over OpenFlow one OS-Ken controller per domain; traffic steering, telemetry consumption, and elasticity decisions run in the controller process
Edge tier	Docker edge-service replicas per domain; a static fleet extended by dynamic replicas during scale-up
Storage tier	one MongoDB replica set per domain; members added and removed through native replication
Telemetry	one aggregator per domain; windowed summaries over a fixed aggregation window of 10 s
Workload driver	open-loop scheduled driver: 24 clients per domain (48 in total), a bounded in-flight window of 1024 requests, a client-side request cap of 300 s, and a 30 s phase-end drain

The testbed runs the two domains on a single host, which keeps the control paths host-local: telemetry delivery and controller-to-controller topology exchange do not traverse the emulated wide-area link, which carries only cross-domain data traffic. The evaluation

workloads reference locally owned content only, so storage reads remain within the replica set of the domain that serves each request, and cross-region storage placement is disabled. Backend selection may also place a request on a peer-domain replica at a hop-cost penalty, so the evaluated service path is not confined to the client’s own domain. Cross-domain placement occurred in all three campaigns: about 2 % of requests with a recorded serving backend in the telemetry delivery and scaling action campaigns and 0.5 % in the readiness admission campaign, concentrating under compute-bound demand, where local compute is most loaded. It is a property of the platform’s selection policy rather than a treatment, and the requests it affects carry the emulated inter-domain latency in their client-side service time. The campaigns therefore measure the coordination interfaces under controlled and repeatable conditions, and no geo-distributed or wide-area performance claim is made.

Each run is an independent experimental unit and starts from the same baseline fleet and configuration. During a run, the platform records the timeline of the demand-to-capacity chain end to end: workload onset, telemetry window completion and controller observation, scaling decision and selected action, provisioning start and completion, application-readiness confirmation, admission to the routing pool, first routed request, and first successful response. The rest of this section defines the workloads that drive these timelines, the outcomes extracted from them, and the design under which each campaign varied exactly one interface.

5.1.2. Demand Model and Campaign Workloads

The demand model varies two axes: the request intensity over time, and the request mix, which determines whether an episode binds the compute tier or the storage tier. The spatial axis, that is, which domain owns the requested content, is held constant in the evaluation, as described above.

A workload is a timed sequence of demand phases. Each phase fixes, for its duration, the offered request intensity, as a per-client request rate and an active-client fraction, and the request mix; the phase that applies the demand shift under test is referred to as the demand episode. Each campaign defined its own workload, calibrated and locked before the campaign started and captured in every run, so that the campaigns do not share a single common workload.

- The telemetry delivery campaign (RQ1) ran a five-phase workload: a 60 s baseline, a 180 s overload episode at a per-client rate of 1.2 requests per second with a co-loaded mix (30 % service pressure, 35 % content lookups, 15 % ranking, and 10 % each of updates and aggregations), a 120 s recovery gap, a 420 s demand drop, and a 420 s idle tail.
- The scaling action campaign (RQ2) ran two separately constructed episodes: a compute-bound episode (600 s at a per-client rate of 1.5, pure service pressure) and a data-access-bound episode (480 s at a per-client rate of 5.0, 90 % content lookups), each with baseline, recovery, and demand-drop phases.

- The readiness admission campaign (RQ3) comprised two parts. The admission timing was measured with an intact readiness-event source, and the consequence of event loss was measured on a baseline followed by a 600 s compute-bound plateau at a per-client rate of 2.0 with pure service pressure, under a fixed and deliberately reduced per-backend CPU quota (0.12) that constrains the headroom of the incumbent fleet.

5.1.3. Metrics and Outcome Definitions

The primary reaction outcome is usable-capacity latency: the time from the onset of the demand episode to the first new capacity serving traffic, reconstructed from the recorded timeline. The time to the first scaling decision is reported as a descriptive outcome only, because the delivery treatment itself shifts when a decision can be taken. The readiness admission campaign additionally reports the segment from the scaling decision to usable capacity, which isolates the admission path.

Service quality is summarised per run through percentiles (p50, p95, p99) of the endpoint latency over the demand episode. Latency percentiles are computed on the served basis, over completed requests, and, where an offered-basis figure is needed, over completed requests and timeouts together, with a timeout counted at the client-side cap of 300 s. Outcome classes are reported distinctly and never merged: offered requests, completed requests, failed requests, timeouts, and client-side cancellations and drops at phase end. Completion is reported on the served basis, completed requests over non-abandoned requests, and on the offered basis, completed requests over offered requests. Unless stated otherwise, each run contributes the mean of its two domains, and any cross-domain asymmetry above the pre-registered symmetry screen is reported.

The campaign-specific outcomes are the following.

- The telemetry delivery campaign reports the delivered fraction, the fraction of produced telemetry windows delivered to the decision engine; the information age at decision, the time between the completion of a window and its use in a decision; the delay in detecting the overload onset; and the number of overload windows missed.
- The scaling action campaign reports the time to recover the bottleneck-specific pressure, the relief obtained in the targeted tier, the compute and storage node-minutes, the integrated running time of the provisioned units per tier, and the number of scaling actions.
- The readiness admission campaign reports the intervals from verified readiness to admission, from admission to the first routed request, and from the first flow to the first successful response; the useful initial request share, the fraction of requests dispatched to a newly admitted backend that complete successfully; the slow-share, the fraction of served requests (completed or timed out) that time out or exceed one second; and the latency and failure share in the transition window

around admission. A backend counts as admitted when it becomes visible in the selector’s eligible pool, not when its registration is recorded.

Control overhead is reported as the CPU utilisation and resident memory of the controller and telemetry processes.

5.1.4. Campaign Designs and Analysis

All three campaigns follow the same discipline of single-variable manipulation. Each campaign varies exactly one interface of the demand-to-capacity chain and holds the other two constant, so that the measured differences are attributable to the interface under test. The policies compared within a campaign are baselines that encode architectural properties, such as a fixed single-tier scaling policy or a periodic discovery mechanism, and not competing products. Each campaign was pre-registered: the treatments, primary outcomes, screening rules, and analysis tests were fixed before data collection. No results are pooled across campaigns; every contrast is computed within its own campaign’s design, and where a research question draws on more than one campaign the campaigns are reported separately.

Tables 5.2 to 5.4 summarise the design of each campaign.

TABLE 5.2. Design of the telemetry delivery campaign (RQ1).

Item	Design
Treatment	Four telemetry delivery semantics over the otherwise identical platform: event-preserving delivery of every completed window (A); event-preserving delivery delayed by a fixed 30 s, without burst replay (B); latest-state polling every 30 s, delivering about one third of the windows (C); and sampled push delivering every third window (D).
Held constant	Aggregation window (10 s), scaling policy and parameters, routing policy, workload, topology, resource limits.
Workload	Baseline (60 s) → overload episode (180 s, rate 1.2 per client, co-loaded mix) → recovery gap (120 s) → demand drop (420 s) → idle tail (420 s).
Experimental unit	Run; outcome percentiles as the per-run mean of the two domains.
Replication	Four arms and seven seed-matched blocks, one run per arm per block (28 runs, seeds 3001–3007), counterbalanced order.
Exclusions	None excluded; all 28 runs passed the per-run platform checks. Reported flags: one delayed replicate above the threefold cross-domain symmetry line; one cancellation outlier in the sampled-push arm; a residual completed-without-status class concentrated in the delayed arm.
Test	Four pre-registered contrasts on the served-basis episode latency p95 (delay A–B, delay D–C, loss A–D, loss B–C): exact Mann–Whitney U (complete enumeration, seven against seven), Cliff’s δ with 95 % bootstrap CI, direction counts; no multiplicity correction.

The reported contrasts follow the test pre-registered for each campaign, with effect sizes reported with confidence intervals and direction counts over matched replicates. Screening rules are applied before the contrasts: runs entering a contrast must pass the campaign’s per-run checks on data-path integrity, harness behaviour, delivery fidelity,

TABLE 5.3. Design of the scaling action campaign (RQ2).

Item	Design
Treatment	Three scaling policies (compute-first, storage-first, and bottleneck-aware selection) each under two demand regimes (compute-bound and data-access-bound), giving six cells. Compute scale-up adds an edge-service replica; storage scale-up activates a prepared reserve in the same domain.
Held constant	Telemetry delivery, routing policy, per-tier resource caps, action budget, cooldowns, backend-admission condition; Tier 1 selective synchronisation and cross-region storage placement disabled.
Workload	Compute-bound episode (600 s, rate 1.5 per client, pure service pressure) or data-access-bound episode (480 s, rate 5.0 per client, 90 % content lookups); each with baseline, recovery, and demand-drop phases.
Experimental unit	Run; per-cell criterion over replicates.
Replication	Six cells and six replicates in six counterbalanced blocks (36 runs; five seed-matched replicates and one cross-seed replicate per cell); 34 valid.
Exclusions	Two runs excluded as documented container memory-cap incidents (kills under the 256 MB and 512 MB caps); per-run screens as above.
Test	Pre-registered cell-level criterion: median of the replicate p95 ratios between the pre-registered pre-activation and post-activation windows, with a 95 % CI excluding 1.0, computed on the seed-matched replicates (five per cell, or fewer where an exclusion applies); the cross-seed replicate reported separately; tier-relief and resource-use outcomes alongside.

TABLE 5.4. Design of the readiness admission campaign (RQ3).

Item	Design
Treatment	Readiness-admission coordination: direct lifecycle notification versus periodic discovery (10 s poll), with a probe fallback variant, each measured with an intact readiness-event source and under total event loss.
Held constant	Readiness criterion (same probe), backend-selection function, routing weights, pool state, workload shape, per-backend CPU quota.
Workload	Admission timing with an intact event source; fault contrast on a baseline followed by a 600 s compute-bound plateau (rate 2.0 per client, pure service pressure) under fixed reduced headroom (per-backend CPU quota 0.12).
Experimental unit	Run; the fault contrast contrasted per block, admission intervals additionally analysed per backend within runs.
Replication	Three admission configurations and two fault cells in six blocks (36 runs, seeds 5201–5206), arm order rotated per block; the admission timing is characterised on separate intact-source campaigns; a supporting robustness set exercises event loss and controller restart (19 runs). Samples are never pooled across these sets.
Exclusions	One block excluded from the fault contrast (no-fault control tail breach); pre-registered health floors on the no-fault cell.
Test	Admission timing: exact Mann–Whitney U on the per-backend admission intervals, stratified by the backend’s start-up class (fast- vs slow-binding). Fault contrast: co-primary pre-registered slow-share contrasts over the full plateau and the onset window (at least 5 pp; medians at least 7 pp), exact Mann–Whitney U of the block contrasts, relief completion on the plateau tail, and boundedness of the lifecycle-only degradation (at most 25 %).

and cross-domain symmetry, and the defined exclusions are removed and reported. Pre-registered criteria that are not met are reported as such, together with the observed evidence and its documented causes.

5.2. Telemetry Delivery Semantics (RQ1)

The campaign compared the three delivery semantics named in Section 1.3 under the overload episode described in Section 5.1, with the latest-state class realised in two lossy forms. Arm A (event-preserving) hands every completed telemetry window to the decision engine exactly once and in source order; arm B (delayed event-preserving) hands the same ordered windows with a fixed, pre-registered delay of 30 s and no burst replay; arm C (latest-state polling) delivers only the most recent completed window at each 30 s poll; and arm D (sampled push) delivers every third completed window immediately and drops the intermediate windows. The aggregation window, scaling policy, routing policy, and workload were held constant across arms. Each arm was run seven times, once in each of seven seed blocks shared by the four arms, giving 28 runs in total, with each run treated as an experimental unit.

The delivered evidence matched this design. Arms A and B received every completed telemetry window (delivered fraction 1.000), while the lossy arms received about one third of them (C 0.325–0.336, D 0.328–0.336), missing on average 55.0 (C) and 49.8 (D) overload windows per run. The mean information age at decision rose from 3.5 s in A to 11.7 s in D, 16.2 s in C and 33.3 s in B, and the detection of the overload onset was delayed in step: a mean of 0.62 s in A, 0.64 s in D, 5.58 s in C and 30.19 s in B.

Later or lossier delivery also delayed the scaling reaction. The median time from the start of the overload episode to usable capacity (Section 5.1) was 28.5 s in A, 59.6 s in B, 79.6 s in C and 83.2 s in D (Table 5.5). The first two steps of the pre-registered ordering ($A < B < C \approx D$) were fully replicated: $A < B$ and $B < C$ in all seven blocks. C and D were not cleanly separated, with D later at the median by 3.6 s ($C < D$ in five of seven blocks). Reaction claims use usable-capacity latency; the time to the first scale-up decision is descriptive only, because delivery timing confounds it by construction.

The user-visible service quality tracked the reaction timing (Table 5.6, Figure 5.1). Incomplete demand evidence degraded the episode endpoint latency p95 by about nine times: the median per-run p95 was 3.7 s in A against 35.0 s in C and 34.5 s in D, and the A–D contrast separated completely, every A replicate lying below every C and D replicate (Cliff’s $\delta = -1.000$, exact $p = 0.0006$). The delayed arm degraded in aggregate — median 10.8 s against A’s 3.7 s ($\delta = -0.959$, exact $p = 0.0012$) — but not uniformly: two of the seven B replicates collapsed to lossy-arm levels (65.3 s and 64.7 s) while the other five remained near A (5.0–12.6 s).

A post-campaign analysis of the run artefacts attributes the spread of the delayed arm to the margin each semantics leaves to the base fleet rather than to the control loop. Across all seven delayed runs the control loop behaved identically: the decision evidence was 30 s old, the first scale-up decision fell between 52.0 s and 60.4 s, and usable capacity between 52.6 s and 63.6 s. The user-visible outcome depends on how much of the surge the base fleet must absorb alone before that capacity arrives — about 29 s in A, 55–62 s in B, and 78–83 s in C and D — and on the testbed’s run-to-run variance: the episode

median compute processing time varied roughly sixfold (16–94 ms) across runs of identical configuration, in every arm. A’s short window absorbs this variance invisibly; the window of C and D is always consumed; B is conditional, its cost materialising only when the slow states of the platform are sampled. The delivery semantics set the margin; the platform state decides the outcome.

Two pre-registered expectations were not reproduced and are reported as such. The full ordering $A < B < C \approx D$ did not hold: B does not sit reliably between A and C, because its collapsed replicates straddle C (B–C $p = 0.209$, direction 5/7). And the sampled-push arm was not strictly worst: C and D were statistically indistinguishable ($p = 0.456$), with the point estimate marginally against C. One delayed replicate showed a domain asymmetry above the campaign’s threefold screening line (domain 2 p95 21.1 s against 4.0 s in domain 1); it was the only run above that line.

Outside the overload episode the arms were indistinguishable: baseline, recovery and demand-drop p95 stayed at about 1.05–1.08 s in every arm. No run recorded a single timeout inside the episode in any arm; the only timeouts in the campaign were the 300 s client-cap tail timeouts during demand drop (0.07–0.12 % of offered requests). Cancellation shares were flat across arms at roughly 5–7 % of offered requests, and a small completed-without-status error class remained largest in B (2.22 % against 0.03–0.42 % elsewhere). A per-bucket check (30 s buckets) confirmed that the episode ordering is not an artefact of aggregating the episode into a single percentile. The degradation is therefore confined to the overload episode, and no arm showed a harness-level collapse.

In sum, the two failure modes of the observation interface were distinguishable and not interchangeable under this workload: loss of intermediate demand evidence was the robust driver of user-visible cost, while delivery delay was costly in proportion to the margin it consumes.

TABLE 5.5. Delivery-semantics arms: usable capacity, episode latency, and outcome classes (medians with min–max ranges over the seven replicates; delivered fraction and timeouts as observed). Usable capacity is the time from episode onset to first new capacity serving traffic; the episode p95 is the served-basis endpoint latency p95 (per-run mean of the two domains).

Arm	Delivered	Usable capacity (s)	Episode p95 (s)	Timeouts	Cancelled (%)
A – event-preserving	1.000	28.5 (25.1–32.9)	3.7 (2.8–5.2)	0	5.6
B – delayed (+30 s)	1.000	59.6 (55.1–61.5)	10.8 (5.0–65.3)	0	6.5
C – latest-state (poll 30 s)	0.325–0.336	79.6 (73.7–81.8)	35.0 (27.0–39.6)	0	5.4
D – sampled push (/3)	0.328–0.336	83.2 (75.1–88.1)	34.5 (30.7–37.6)	0	6.1

TABLE 5.6. Pre-registered primary contrasts on the episode endpoint latency p95 (served basis; per-run mean of the two domains). Exact Mann-Whitney p -values (complete enumeration, $n = 7$ per arm); Cliff’s δ computed as $P(x > y) - P(x < y)$ with 95 % bootstrap CI; direction = blocks (of seven) consistent with the predicted order. No multiplicity correction (four pre-registered edges).

Contrast	Medians (s)	Exact p	Cliff’s δ [95 % CI]	Direction
Delay A–B	3.73 vs 10.77	0.0012	−0.959 [−1.000, −0.755]	7/7
Delay D–C	34.50 vs 35.04	0.456	−0.265 [−0.837, 0.388]	2/7
Loss A–D	3.73 vs 34.50	0.0006	−1.000 [−1.000, −1.000]	7/7
Loss B–C	10.77 vs 35.04	0.209	−0.429 [−1.000, 0.429]	5/7

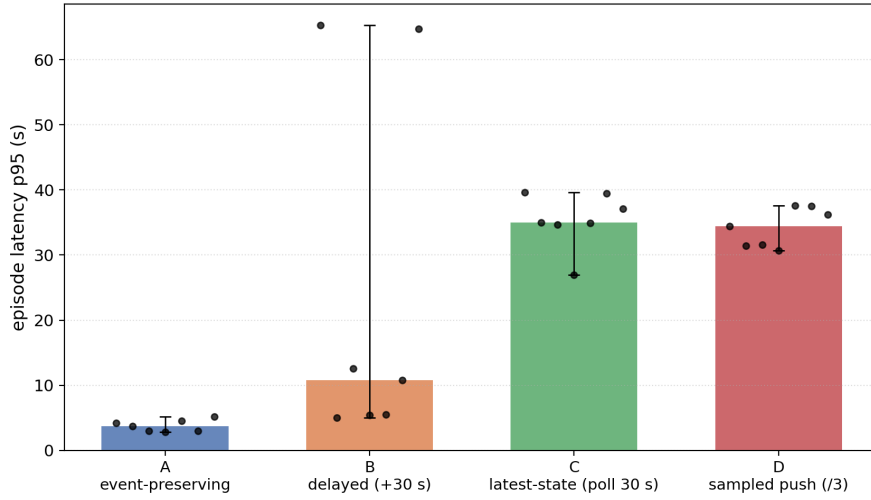


FIGURE 5.1. Episode endpoint latency p95 by delivery semantics (served basis; per-run mean of the two domains; seven replicates per arm). Bars show the median across replicates, whiskers the min–max range, and dots the individual runs; the two highest delayed points are the replicates where the base fleet’s absorption margin was consumed.

5.3. Bottleneck-Aware Scaling Action (RQ2)

5.4. Readiness Admission to Traffic (RQ3)

The admission rules designed for RQ3 (Table 5.4) were evaluated over the same verified-readiness state, with the readiness criterion, the backend-selection function, the workload, and the resource limits held fixed, leaving the admission rule as the only variable. The direct lifecycle notification admits a backend when its readiness event arrives and re-checks nothing — its 130 s fallback timer sits beyond the 120 s probe bound and is inert by construction. The probe fallback variant combines the same event with a bounded 20 s fallback probe, and periodic discovery admits the verified state on a fixed 10 s poll. A backend counts as admitted when it becomes visible in the selector’s eligible pool, and in the event-loss part of the campaign, the controller drops every readiness event. Two evidence lines feed this section: the intact-source measurements quantify the admission

delay and its propagation, and the event-loss part prices the loss of the readiness event under a fixed per-backend CPU quota (0.12), deliberately reduced so that the incumbent fleet carries the thin spare capacity that the research question conditions on (36 runs in six seed blocks, per-client rate 2.0).

Under an intact event source the two mechanisms differed by a bounded quantization. The per-backend interval from verified readiness to admission had a median of 0.001 s under direct lifecycle notification against 6.984 s under periodic discovery ($d = -1.000$; earlier in every backend start-up class, fast- and slow-binding alike), and the quantum followed the configured poll period, growing from 9.6 s to 15.2 s when the period grew from 10 s to 15 s (Table 5.7). The differential propagated along the recorded chain: the interval from provisioning to the first successful response was about six seconds shorter (11.27 s against 17.70 s, exact $p = 0.0005$), and the elasticity loop closed about four seconds sooner (scaling decision to usable capacity 2.17 s against 6.01 s, exact $p = 0.0022$). The handover itself was not the treatment: once admitted, the interval from admission to the first routed request measured 0.4–1.2 s on both mechanisms (a pre-fix servability defect had produced a lag of about ten seconds; the reported values are post-fix), the useful initial request share was 1.000, and the incumbent relief magnitudes did not separate (exact $p = 0.209$). No user-visible cost of the seconds-level delay appeared in any tested healthy regime.

When the readiness event is lost under reduced headroom, the rules diverge in kind rather than in degree (Table 5.8). Under direct lifecycle notification no backend was admitted in any of the six blocks (zero admissions, and no request served by the new capacity, in 6/6; 32–35 candidates per block were abandoned and torn down), while both re-deriving rules recovered — the probe fallback variant after its fallback window (admission about 21 s after provisioning), periodic discovery within one poll period, and on the re-anchored ordering test, measured from each backend’s provisioning rather than from the plateau onset (analysis-only), periodic discovery recovered sooner: provisioning to admission in all six blocks, provisioning to first serve in five of six plus one sub-second tie. The consequence was visible to users: the direct lifecycle configuration degraded to 8.3–14.4 % plateau slow share, concentrated in the transition window around admission (onset window 29.7–39.0 %), a full-plateau contrast of 6.2–10.9 pp against the recovering configurations (median 7.64 pp, exact $p = 0.0040$, Cliff’s $\delta = 1.0$) and an onset contrast of 25.1–30.8 pp (median 28.0 pp), both pre-registered co-primary windows and both reproduced in every block entering the contrast (5/5). The degradation was bounded and predominantly onset-concentrated: the lifecycle-only configuration’s plateau slow median was 10.6 % under a pre-registered ceiling of 25 %, the recovering configurations completed relief on the plateau tail in five of the six blocks (tails of at most 2 pp; the sixth block was excluded from the fault contrast, its no-fault control tail breaching the 2 pp floor at 4.08 %), and one block retained an 11.4 % residual tail, so the transient is not uniform across blocks. What the contrast isolates is lost admission, not delay: under direct

lifecycle notification the capacity was provisioned, ready, and not selectable, whereas the delayed admissions of the recovering configurations sufficed for relief to complete.

Two qualifiers bound this reading. First, the campaign’s pre-registered conjunction (the headline contrast, its relief-ordering component, and the no-fault health floors) was not fully met (overall pass = false, gated on the no-fault health floors alone): the headline contrast is met, the relief-ordering component is met on its re-anchored test alone (the as-run onset anchor measured the phase boundary), and the no-fault health floors were met in 16 of 18 no-fault runs, with two marginal single-window breaches (a failure-or-timeout share of 1.19 % and one 30 s tail window at 4.29 % slow share) that do not touch the between-configuration contrast. Second, the consequence is demonstrated for total event loss with reduced headroom under demand escalation; selective loss, message duplication or reordering, and controller restart are outside the tested scope, and no claim is made about them.

In sum, readiness-admission coordination is at once a latency choice — about seven seconds of quantization while everything works — and a reliability choice: it decides whether a ready backend’s capacity is delayed or lost when a readiness notification is lost and spare capacity is already low.

TABLE 5.7. Admission-chain intervals and healthy-regime outcomes (intact readiness-event source; direct lifecycle notification versus periodic discovery). Medians over runs and per-backend intervals as marked; exact Mann–Whitney p -values as pre-registered.

Interval	Direct	Periodic	Test
Verified readiness→admission (s)	0.001	6.984	$d = -1.000$; earlier in every start-up class
Provisioning→first successful response (s)	11.27	17.70	exact $p = 0.0005$, $\delta = -0.594$
Scaling decision→usable capacity (s)	2.17	6.01	exact $p = 0.0022$, $d = -1.000$
Admission→first routed request (s)	0.4–1.2	0.4–1.2	confirmatory; not separated
Incumbent relief (per-run median, pp)	16.0	19.3	exact $p = 0.209$; not separated

TABLE 5.8. Readiness-admission configurations under total event loss (event-loss part of the RQ3 campaign; six blocks, plateau pooled). Admitted = visibility in the selector’s eligible pool; abandoned candidates in parentheses. Plateau tail = slow share over the last 300 s of the plateau; *block 4 is excluded from the fault contrast, its no-fault control tail (4.08 %) breaching the 2 pp floor.

Configuration	Re-derivation	Admitted (abandoned)	Plateau slow (%)	Plateau tail (%)
Direct lifecycle notification	none; fallback inert	0 (32–35)	8.3–14.4	1.0–1.3, except b2: 11.4
Probe fallback variant	20 s fallback probe	4–6 (0)	1.9–3.6	≤1.5
Periodic discovery	10 s poll	4–5 (0)	2.0–5.0	≤1.4, except b4*: 4.08

5.5. Discussion: Synthesis Across Interfaces

CHAPTER 6

Conclusions and Future Work

6.1. Conclusions

6.2. Research Contributions

6.3. Limitations

6.4. Future Research Directions

[This page is intentionally left blank.]

References

- [1] C. Qu, R. N. Calheiros, and R. Buyya, “Auto-scaling web applications in clouds: A taxonomy and survey,” *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–33, Jul. 2018. DOI: 10.1145/3148149 [Online]. Available: <https://doi.org/10.1145/3148149>
- [2] K. L. Bogdanov, W. Reda, G. Q. Maguire Jr., D. Kostić, and M. Canini, “Fast and accurate load balancing for geo-distributed storage systems,” in *Proceedings of the ACM Symposium on Cloud Computing*, Carlsbad CA USA, Oct. 2018, pp. 386–400. DOI: 10.1145/3267809.3267820 [Online]. Available: <https://doi.org/10.1145/3267809.3267820>
- [3] A. Llorens-Carrodegua, I. Leyva-Pupo, C. Cervelló-Pastor, L. Piñeiro, and S. Siddiqui, “An sdn-based solution for horizontal auto-scaling and load balancing of transparent vnf clusters,” *Sensors*, vol. 21, no. 24, p. 8283, Dec. 2021. DOI: 10.3390/s21248283 [Online]. Available: <https://doi.org/10.3390/s21248283>
- [4] International Telecommunication Union, “Facts and figures 2025: Internet traffic,” ITU, Oct. 2025. Accessed: Jul. 13, 2026. [Online]. Available: <https://www.itu.int/itu-d/reports/statistics/2025/10/15/ff25-internet-traffic/>
- [5] D. Gurung, S. M. Z. U. Rashid, Z. u. Abdeen, and S. Rath, “Cloud revolution: Tracing the origins and rise of cloud computing,” in *2026 IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC)*, Las Vegas, NV, USA, Jan. 2026, pp. 1100–1106. DOI: 10.1109/CCWC67433.2026.11393790 [Online]. Available: <https://doi.org/10.1109/CCWC67433.2026.11393790>
- [6] T. Gaudiaut, *Mobile accounts for nearly 60 percent of web traffic*, Statista Chart of the Day, Data from StatCounter. Licensed under CC BY-ND 3.0, Mar. 2026. Accessed: Jul. 19, 2026. [Online]. Available: <https://www.statista.com/chart/35951/share-of-mobile-device-website-traffic-worldwide-since-2010/>
- [7] K. Cao, Y. Liu, G. Meng, and Q. Sun, “An overview on edge computing research,” *IEEE Access*, vol. 8, pp. 85 714–85 728, 2020. DOI: 10.1109/ACCESS.2020.2991734 [Online]. Available: <https://doi.org/10.1109/ACCESS.2020.2991734>
- [8] M. Satyanarayanan, “The emergence of edge computing,” *Computer*, vol. 50, no. 1, pp. 30–39, Jan. 2017. DOI: 10.1109/MC.2017.9 [Online]. Available: <https://doi.org/10.1109/MC.2017.9>
- [9] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, “Resource scheduling in edge computing: A survey,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2131–2165, 2021. DOI: 10.1109/COMST.2021.3106401 [Online]. Available: <https://doi.org/10.1109/COMST.2021.3106401>

- [10] M. Breitbach, D. Schäfer, J. Edinger, and C. Becker, “Context-aware data and task placement in edge computing environments,” in *2019 IEEE International Conference on Pervasive Computing and Communications (PerCom)*, Kyoto, Japan, Mar. 2019, pp. 1–10. DOI: 10.1109/PERCOM.2019.8767386 [Online]. Available: <https://doi.org/10.1109/PERCOM.2019.8767386>
- [11] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–106, Mar. 2004. DOI: 10.2307/25148625 [Online]. Available: <https://doi.org/10.2307/25148625>
- [12] J. vom Brocke, A. Hevner, and A. Maedche, “Introduction to design science research,” in *Progress in IS*, 2020, pp. 1–13, ISBN: 9783030467807. DOI: 10.1007/978-3-030-46781-4_1 [Online]. Available: https://doi.org/10.1007/978-3-030-46781-4_1
- [13] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, Dec. 2007. DOI: 10.2753/MIS0742-1222240302 [Online]. Available: <https://doi.org/10.2753/MIS0742-1222240302>
- [14] F. Liu, G. Tang, Y. Li, Z. Cai, X. Zhang, and T. Zhou, “A survey on edge computing systems and tools,” *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1537–1562, 2019. DOI: 10.1109/JPROC.2019.2920341 [Online]. Available: <https://doi.org/10.1109/JPROC.2019.2920341>
- [15] A. Nain, S. Sheikh, M. Shahid, and R. Malik, “Resource optimization in edge and sdn-based edge computing: A comprehensive study,” *Cluster Computing*, vol. 27, no. 5, pp. 5517–5545, Feb. 2024. DOI: 10.1007/s10586-023-04256-8 [Online]. Available: <https://doi.org/10.1007/s10586-023-04256-8>
- [16] A. C. Baktir, A. Ozgovde, and C. Ersoy, “How can edge computing benefit from software-defined networking: A survey, use cases, and future directions,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2359–2391, 2017. DOI: 10.1109/COMST.2017.2717482 [Online]. Available: <https://doi.org/10.1109/COMST.2017.2717482>
- [17] Q. Waseem, W. Isnii Sofiah Wan Din, A. Aminuddin, M. Hussain Mohammed, and R. F. Alfa Aziza, “Software-defined networking (sdn): A review,” in *2022 5th International Conference on Information and Communications Technology (ICOIACT)*, Yogyakarta, Indonesia, Aug. 2022, pp. 30–35. DOI: 10.1109/ICOIACT55506.2022.9972067 [Online]. Available: <https://doi.org/10.1109/ICOIACT55506.2022.9972067>
- [18] MongoDB, Inc., *Replication*, MongoDB Manual, 2026. Accessed: Sep. 13, 2026. [Online]. Available: <https://www.mongodb.com/docs/manual/replication/>
- [19] N. Yaseen, “From counters to telemetry: A survey of programmable network-wide monitoring,” *Network*, vol. 5, no. 3, p. 38, Sep. 2025. DOI: 10.3390/network5030038 [Online]. Available: <https://doi.org/10.3390/network5030038>

- [20] C.-K. Huang and G. Pierre, “Adappf: Self-adaptive scrape interval for monitoring in geo-distributed cluster federations,” in *2023 IEEE Symposium on Computers and Communications (ISCC)*, Gammarth, Tunisia, Jul. 2023, pp. 417–423. DOI: 10.1109/ISCC58397.2023.10218080 [Online]. Available: <https://doi.org/10.1109/ISCC58397.2023.10218080>
- [21] H. Zhou and C. H. Yong, “Implement hpa for nginx service using custom metrics under kubernetes framework,” *IEEE Access*, vol. 12, pp. 189 722–189 734, 2024. DOI: 10.1109/ACCESS.2024.3509876 [Online]. Available: <https://doi.org/10.1109/ACCESS.2024.3509876>
- [22] M.-H. Hung, C.-C. Teng, C.-P. Chuang, C.-S. Hsu, J.-W. Gong, and M.-C. Chen, “A sdn controller monitoring architecture for 5g backhaul networks,” in *2022 23rd Asia-Pacific Network Operations and Management Symposium (APNOMS)*, Takamatsu, Japan, Sep. 2022, pp. 1–4. DOI: 10.23919/APNOMS56106.2022.9919988 [Online]. Available: <https://doi.org/10.23919/APNOMS56106.2022.9919988>
- [23] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, “A survey on observability of distributed edge & container-based microservices,” *IEEE Access*, vol. 10, pp. 86 904–86 919, 2022. DOI: 10.1109/ACCESS.2022.3193102 [Online]. Available: <https://doi.org/10.1109/ACCESS.2022.3193102>
- [24] M. I. S. El Mrabet, A. Malki, and M. Malki, “A comprehensive survey and taxonomy of autoscaling techniques in cloud and edge environments,” in *2025 Fourth International Conference on Theoretical and Applicative Aspects of Computer Science (ICTAACS)*, Skikda, Algeria, Dec. 2025, pp. 1–8. DOI: 10.1109/ICTAACS69003.2025.11399335 [Online]. Available: <https://doi.org/10.1109/ICTAACS69003.2025.11399335>
- [25] S. Taherizadeh and V. Stankovski, “Auto-scaling applications in edge computing,” in *Proceedings of the International Conference on Big Data and Internet of Thing*, London United Kingdom, Dec. 2017, pp. 158–163. DOI: 10.1145/3175684.3175709 [Online]. Available: <https://doi.org/10.1145/3175684.3175709>
- [26] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, “Machine learning-based scaling management for kubernetes edge clusters,” *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 958–972, Mar. 2021. DOI: 10.1109/TNSM.2021.3052837 [Online]. Available: <https://doi.org/10.1109/TNSM.2021.3052837>
- [27] L. Ju, P. Singh, and S. Toor, “Proactive autoscaling for edge computing systems with kubernetes,” in *Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing Companion*, Leicester United Kingdom, Dec. 2021, pp. 1–8. DOI: 10.1145/3492323.3495588 [Online]. Available: <https://doi.org/10.1145/3492323.3495588>
- [28] K. Cheng, S. Zhang, C. Tu, X. Shi, Z. Yin, S. Lu, Y. Liang, and Q. Gu, “Proscale: Proactive autoscaling for microservice with time-varying workload at the edge,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1294–

- 1312, Apr. 2023. DOI: 10.1109/TPDS.2023.3238429 [Online]. Available: <https://doi.org/10.1109/TPDS.2023.3238429>
- [29] Y. Li, H. Zhang, W. Tian, and H. Ma, “Joint optimization of auto-scaling and adaptive service placement in edge computing,” in *2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS)*, Beijing, China, Dec. 2021, pp. 923–930. DOI: 10.1109/ICPADS53394.2021.00121 [Online]. Available: <https://doi.org/10.1109/ICPADS53394.2021.00121>
- [30] A. Y. Nikraves, S. A. Ajila, and C.-H. Lung, “The impact of database layer on auto-scaling decisions in a 3-tier web services cloud resource provisioning,” in *2017 IEEE 41st Annual Computer Software and Applications Conference (COMPSAC)*, Turin, Jul. 2017, pp. 401–406. DOI: 10.1109/COMPSAC.2017.235 [Online]. Available: <https://doi.org/10.1109/COMPSAC.2017.235>
- [31] J. Dogani, R. Namvar, and F. Khunjush, “Auto-scaling techniques in container-based cloud and edge/fog computing: Taxonomy and survey,” *Computer Communications*, vol. 209, pp. 120–150, Sep. 2023. DOI: 10.1016/j.comcom.2023.06.010 [Online]. Available: <https://doi.org/10.1016/j.comcom.2023.06.010>
- [32] S. Xie, J. Wang, B. Li, Z. Zhang, D. Li, and P. C. K. Hung, “Pbscaler: A bottleneck-aware autoscaling framework for microservice-based applications,” *IEEE Transactions on Services Computing*, vol. 17, no. 2, pp. 604–616, Mar. 2024. DOI: 10.1109/TSC.2024.3376202 [Online]. Available: <https://doi.org/10.1109/TSC.2024.3376202>
- [33] C. Li, J. Bai, Y. Ge, and Y. Luo, “Heterogeneity-aware elastic provisioning in cloud-assisted edge computing systems,” *Future Generation Computer Systems*, vol. 112, pp. 1106–1121, Nov. 2020. DOI: 10.1016/j.future.2020.06.022 [Online]. Available: <https://doi.org/10.1016/j.future.2020.06.022>
- [34] A. A. Neghabi, N. Jafari Navimipour, M. Hosseinzadeh, and A. Rezaee, “Load balancing mechanisms in the software defined networks: A systematic and comprehensive review of the literature,” *IEEE Access*, vol. 6, pp. 14 159–14 178, 2018. DOI: 10.1109/ACCESS.2018.2805842 [Online]. Available: <https://doi.org/10.1109/ACCESS.2018.2805842>
- [35] M. R. Belgaum, S. Musa, M. M. Alam, and M. M. Su’ud, “A systematic review of load balancing techniques in software-defined networking,” *IEEE Access*, vol. 8, pp. 98 612–98 636, 2020. DOI: 10.1109/ACCESS.2020.2995849 [Online]. Available: <https://doi.org/10.1109/ACCESS.2020.2995849>
- [36] A. Wang, Z. Zha, Y. Guo, and S. Chen, “Software-defined networking enhanced edge computing: A network-centric survey,” *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1500–1519, Aug. 2019. DOI: 10.1109/JPROC.2019.2924377 [Online]. Available: <https://doi.org/10.1109/JPROC.2019.2924377>
- [37] R. C. Sofia, D. Dykeman, P. Urbanetz, A. Galal, and D. A. Dave, “Dynamic, context-aware cross-layer orchestration of containerized applications,” *IEEE Access*, vol. 11,

- pp. 93 129–93 150, 2023. DOI: 10.1109/ACCESS.2023.3307026 [Online]. Available: <https://doi.org/10.1109/ACCESS.2023.3307026>
- [38] H. Peng, Q. Ye, and X. S. Shen, “Sdn-based resource management for autonomous vehicular networks: A multi-access edge computing approach,” *IEEE Wireless Communications*, vol. 26, no. 4, pp. 156–162, Aug. 2019. DOI: 10.1109/MWC.2019.1800371 [Online]. Available: <https://doi.org/10.1109/MWC.2019.1800371>
- [39] E. Saurez, H. Gupta, A. Daglis, and U. Ramachandran, “Oneedge: An efficient control plane for geo-distributed infrastructures,” in *Proceedings of the ACM Symposium on Cloud Computing*, Seattle WA USA, Nov. 2021, pp. 182–196. DOI: 10.1145/3472883.3487008 [Online]. Available: <https://doi.org/10.1145/3472883.3487008>
- [40] M. A. Tamiru, G. Pierre, J. Tordsson, and E. Elmroth, “Mck8s: An orchestration platform for geo-distributed multi-cluster environments,” in *2021 International Conference on Computer Communications and Networks (ICCCN)*, Athens, Greece, Jul. 2021, pp. 1–10. DOI: 10.1109/ICCCN52240.2021.9522318 [Online]. Available: <https://doi.org/10.1109/ICCCN52240.2021.9522318>
- [41] H. Tabatabaee Malazi, S. R. Chaudhry, A. Kazmi, A. Palade, C. Cabrera, G. White, and S. Clarke, “Dynamic service placement in multi-access edge computing: A systematic literature review,” *IEEE Access*, vol. 10, pp. 32 639–32 688, 2022. DOI: 10.1109/ACCESS.2022.3160738 [Online]. Available: <https://doi.org/10.1109/ACCESS.2022.3160738>